CSE D 501

More classification models and ensemble methods

Taylor Kessler Faulkner

University of Washington

October 22, 2025

Today’s Class

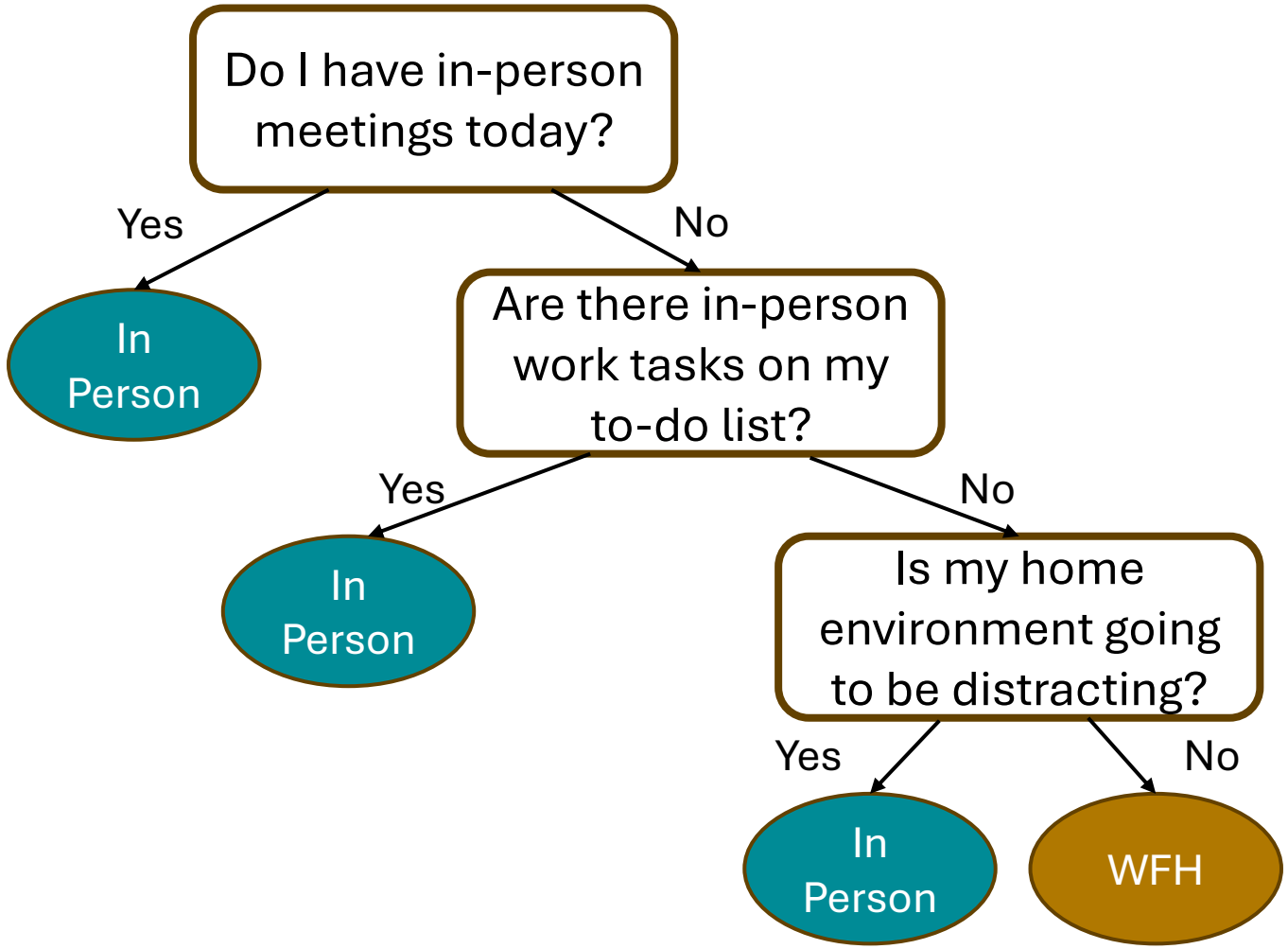
- Decision Trees
- Break
- Ensemble Methods
- Break
- K-nearest neighbors and Kernel Method

Parametric vs non-parametric models

- Parametric models
  - Fixed number of parameters
  - Assume relationships between variables
    - Example: linear relationship for linear regression
  - Examples: Linear Regression, Logistic Regression, Linear SVMs
- Non-parametric models
  - Number of parameters depends on number of samples (size of dataset)
  - Specific functional forms not required
  - We'll cover a few of these:
    - Decision Trees
    - K-Nearest Neighbors
    - Non-linear SVMs using kernels

Decision Trees

- Question: should I work in-office or from home today?
- In life, we might decide this question with a flow chart
- In ML, these are referred to as **decision trees**



RESPIRATORY ILLNESS FLOWCHART

September 5, 2025 / www.ehs.washington.edu

UW Medicine medical facility and other healthcare personnel follow UW Medicine or site-specific protocols.

You are experiencing respiratory virus symptoms.
OR
You were exposed to someone with a respiratory illness.

Do you have symptoms?

YES

STAY HOME AND AWAY FROM OTHERS.

Do not go to work or class.
Stay at home except when seeking medical care.
Consider getting tested to decide on treatment options or whether to be around others, especially for those at risk for severe illness.

Follow CDC guidance to help prevent the spread of respiratory viruses.

You are strongly encouraged to notify others you may have exposed.

If you are UW personnel and believe your illness was due to a workplace exposure, please submit an OARS incident report.[1]

RETURN TO NORMAL ACTIVITIES WHEN:[2]

During the past 24 hours,
Your symptoms have significantly improved and do not interfere with job duties.
AND
You have been fever-free for at least 24 hours without using fever-reducing medications.

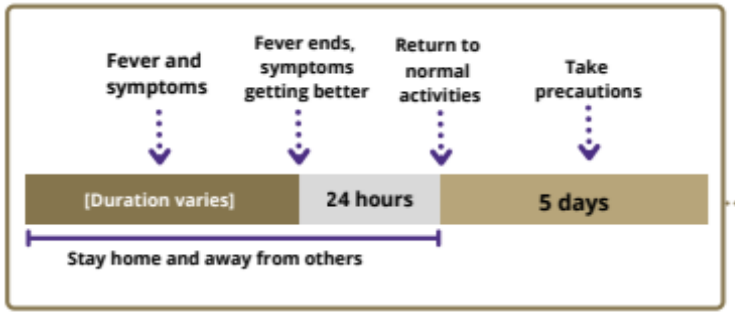
NO

YOU CAN RETURN TO WORK AND CLASS.

If symptoms develop, stay home and follow CDC guidance to help prevent the spread of respiratory viruses.

TAKE PRECAUTIONS FOR THE NEXT 5 DAYS.

(Noted at right).



TAKE THESE PRECAUTIONS WHEN RETURNING TO NORMAL ACTIVITIES[3]

WEAR A MASK

to help prevent the spread of respiratory viruses.

PRACTICE GOOD HYGIENE

by covering sneezes and coughs, washing your hands often, and cleaning common surfaces.

OPEN WINDOWS

(when possible), gather outdoors, and use well-ventilated spaces.

PRACTICE PHYSICAL DISTANCING

between yourself and others when possible.

STAY UP TO DATE

on immunizations that are recommended for you.

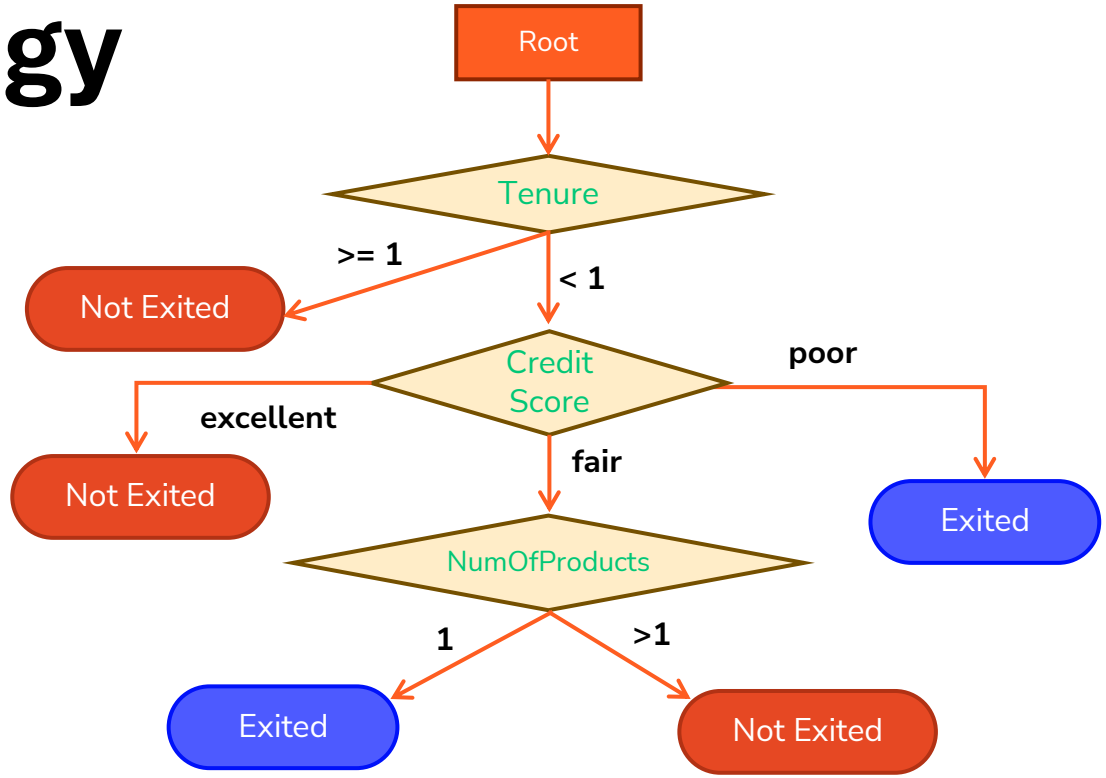
[1] *If a supervisor believes their workforce is experiencing an outbreak (more than 10% of personnel are out sick with the same illness), contact EH&S for assistance.*
[2] *You do **not** need a negative test before returning to normal activities if the other conditions specified here have been met.*
[3] *Keep in mind you may still be able to spread the virus that made you sick, even if you are feeling better. You are likely to be less contagious at this time, depending on factors such as how long you were sick or how sick you were. If you develop a fever or you start to feel worse after you have gone back to normal activities, follow CDC guidance.*

Questions? Contact UW Environmental Health & Safety (EH&S) at ehsdept@uw.edu or (206) 543-7262.
Visit www.ehs.washington.edu/respiratory-illness/respiratory-illness-and-exposure-guidance for a text version.

ENVIRONMENTAL HEALTH & SAFETY
UNIVERSITY of WASHINGTON

Decision Tree Terminology

- Nodes
  - Represent features or decisions
- Branch/Internal nodes
  - Splits feature values
- Leaf node
  - Decision (a class value)
- Root node
  - The top node, where all data starts

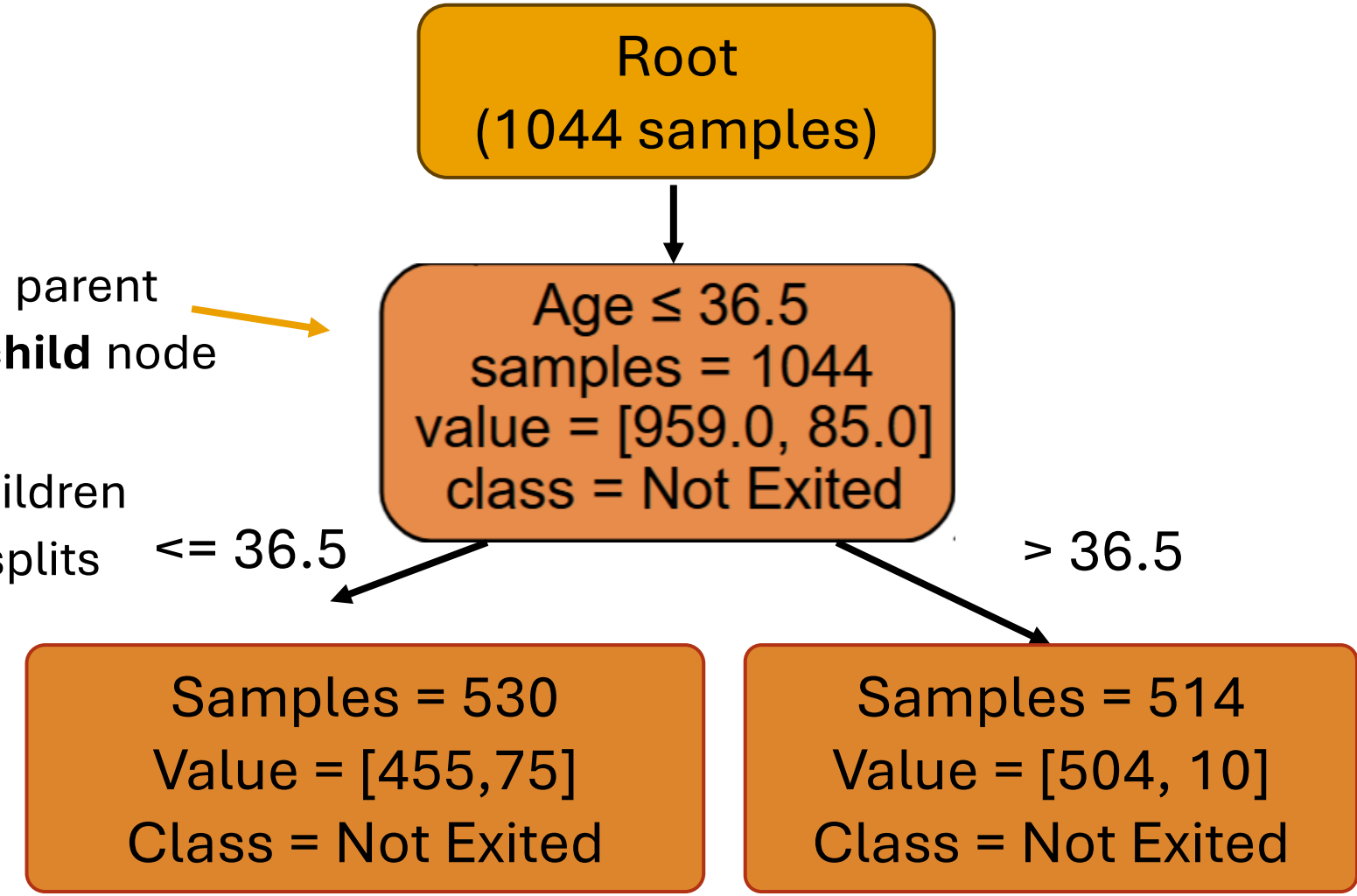


Age ≤ 36.5 → Feature and split (Age <= 36.5, Age > 36.5)
samples = 1044 → Number of samples classified by node (1044)
value = [959.0, 85.0] → How many samples in each class (959 not exited, 85 exited)
class = Not Exited → Majority class of all samples (not exited)

Decision Tree Terminology

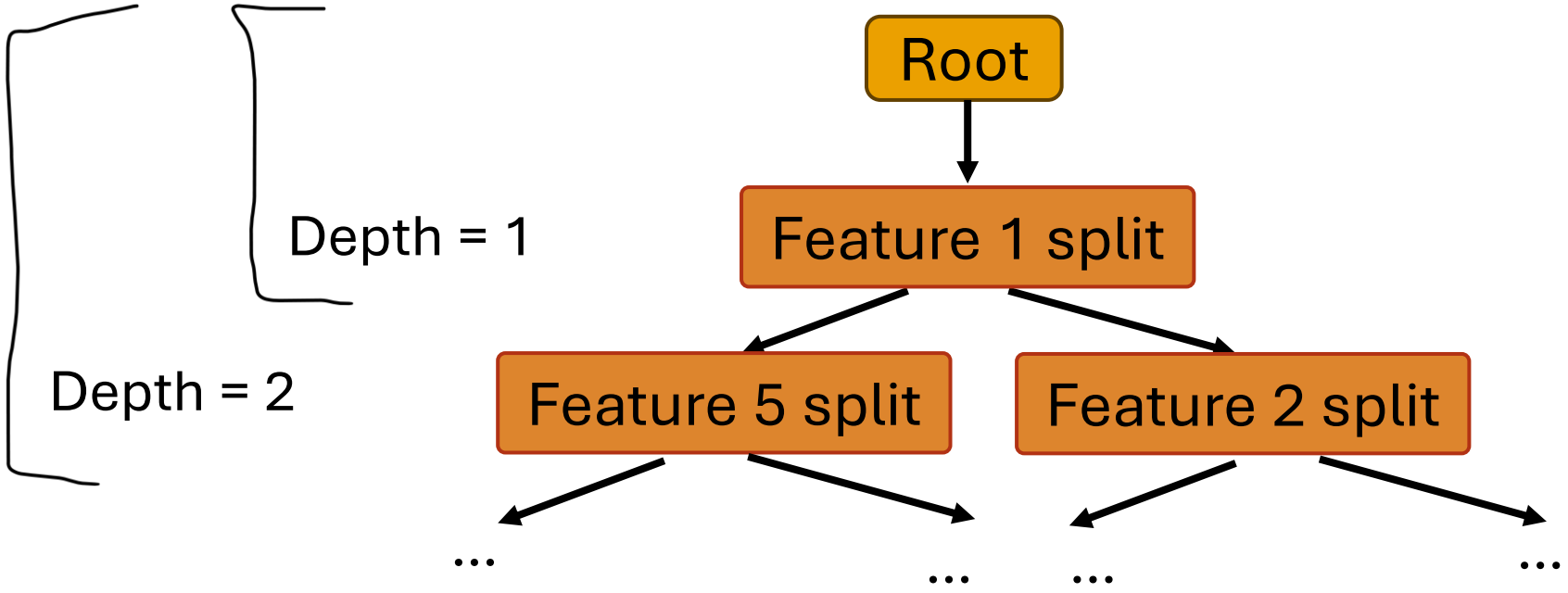
- Decision stump: 1 level

- **Parent**
  - All nodes except the root have a parent
  - The node that split to form the **child** node
- **Child**
  - All nodes except leaves have children
  - The nodes that a parent nodes splits into



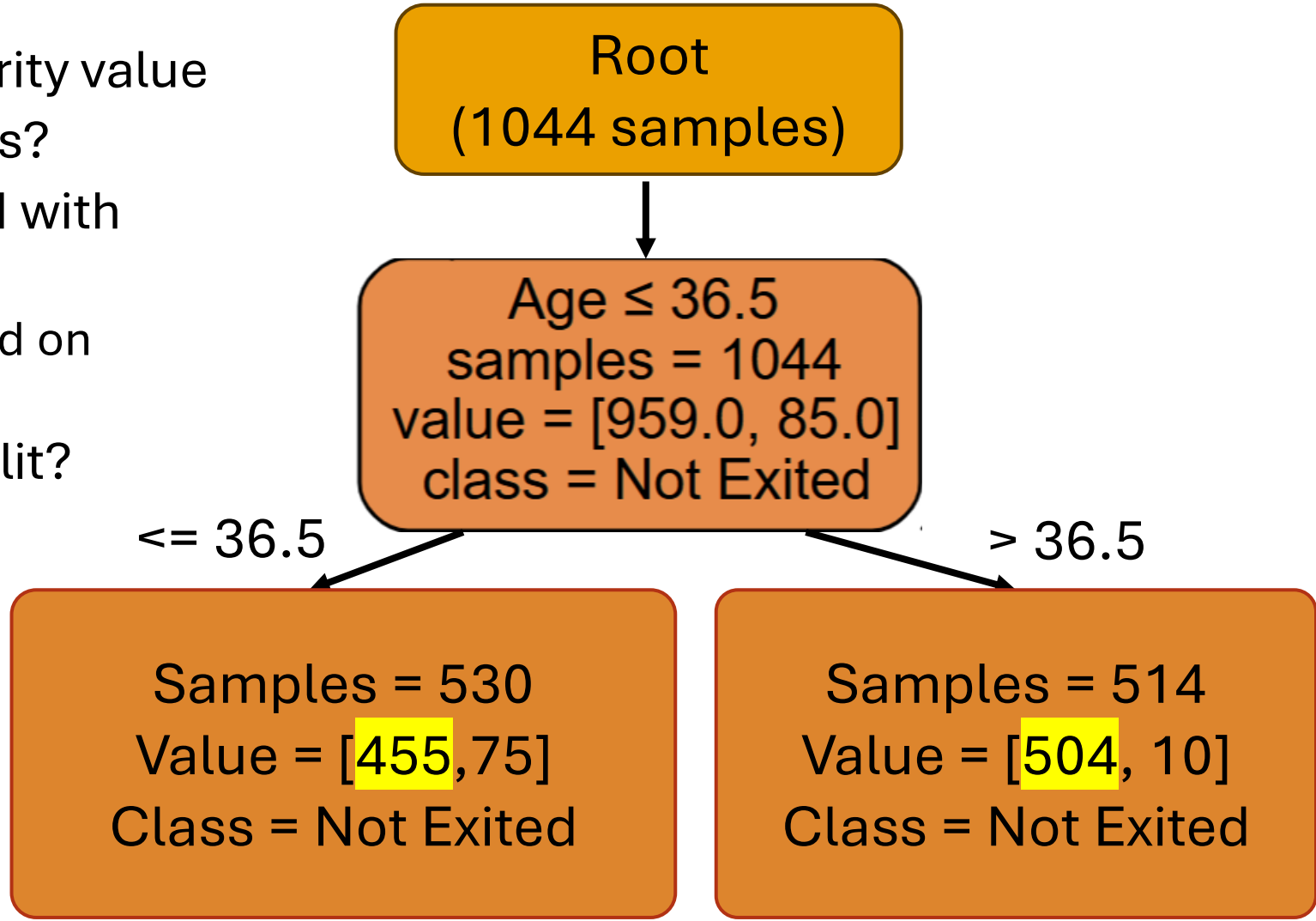
Decision Tree Terminology

- Decision trees have **depth**



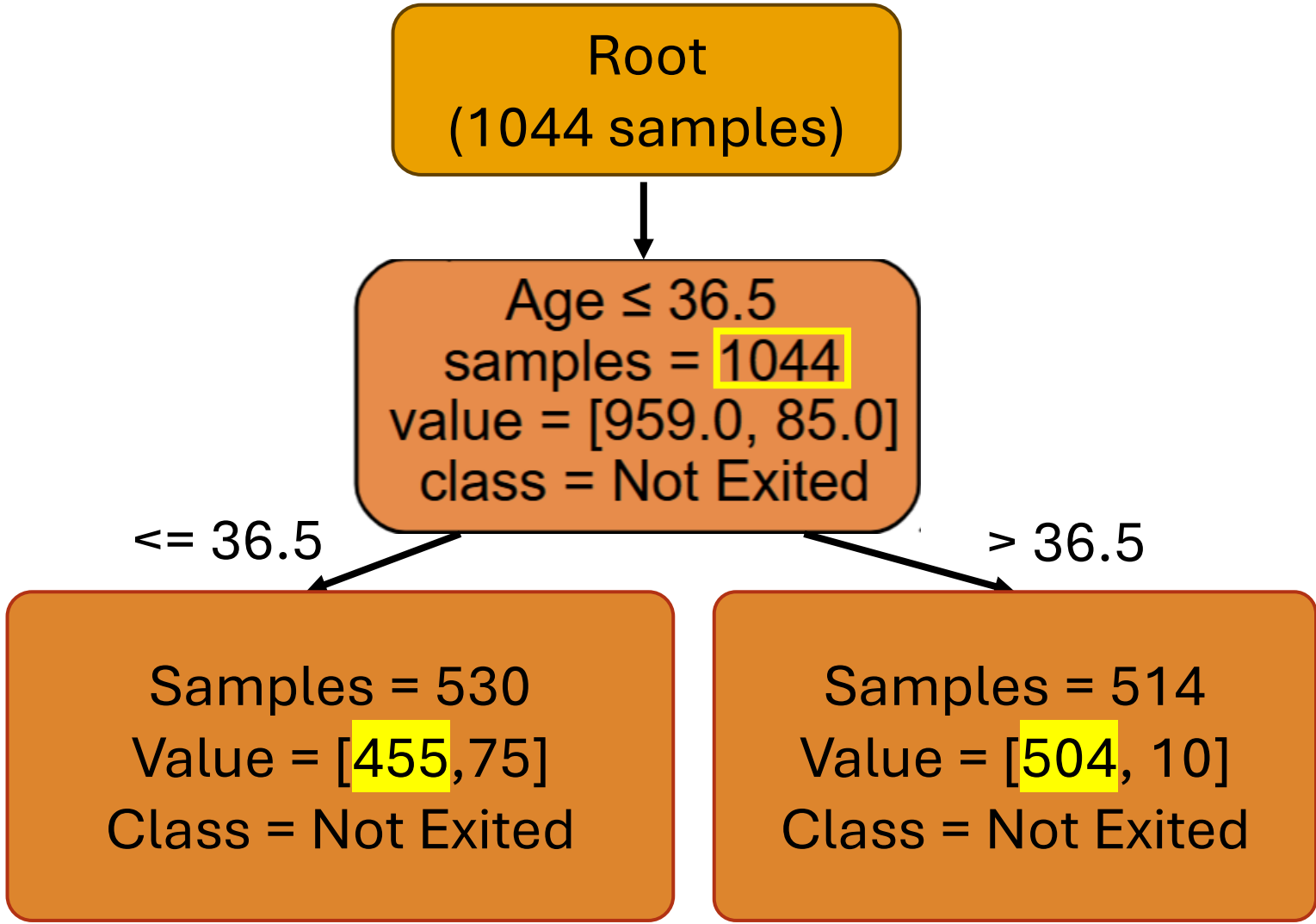
Decision Trees

- For every leaf node, set $\hat{y} =$ majority value
- How do decision trees form nodes?
- Choose the feature and threshold with the **optimal split**
  - **Split**: division into children based on thresholded feature values
- How can we define an optimal split?



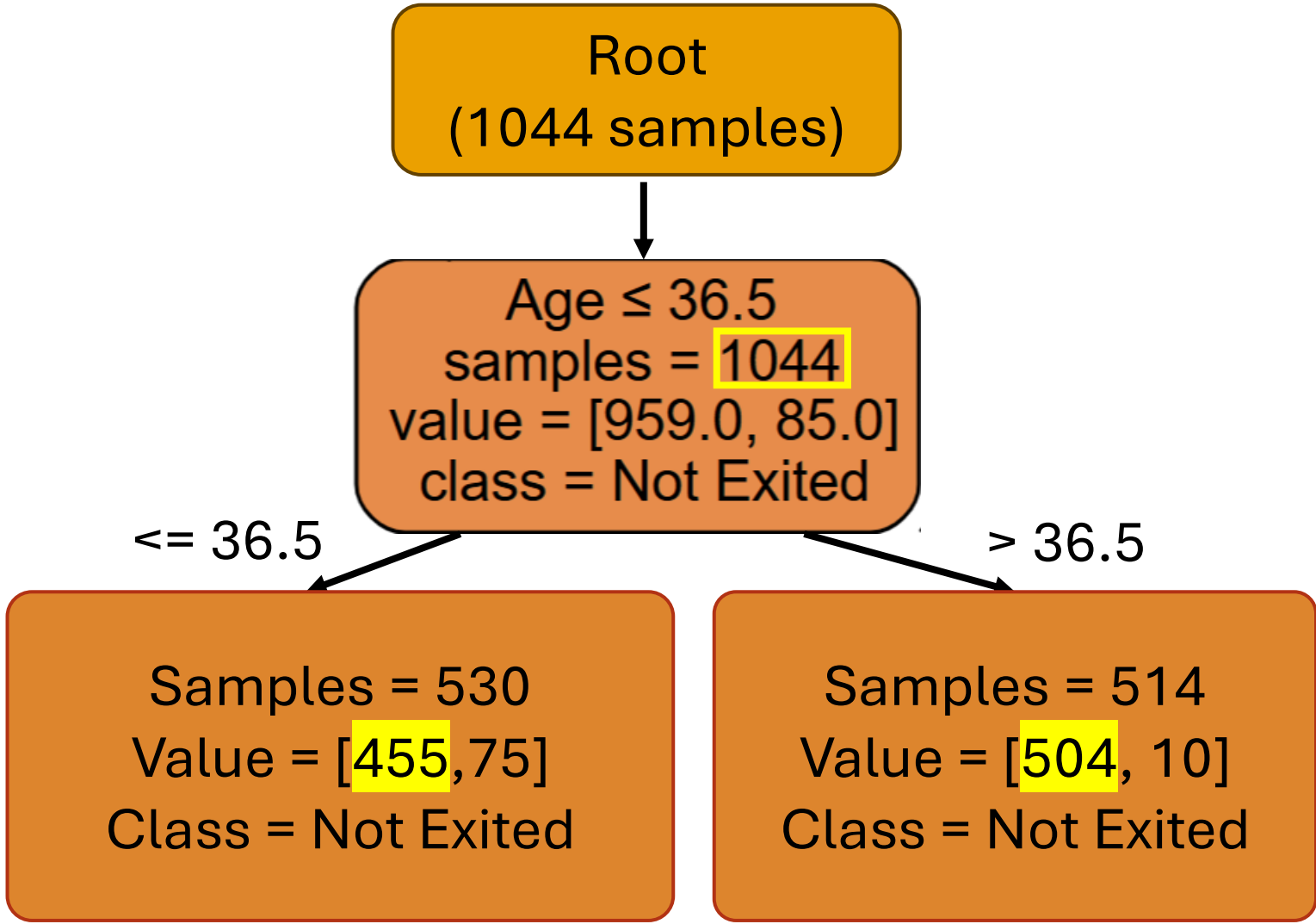
Choosing features and thresholds

- Suppose we are at node i
- Let n_i be the number of samples that reach node i
- We want to split these samples into a left branch and right branch to minimize error
  - Question: why two branches? Why not more?



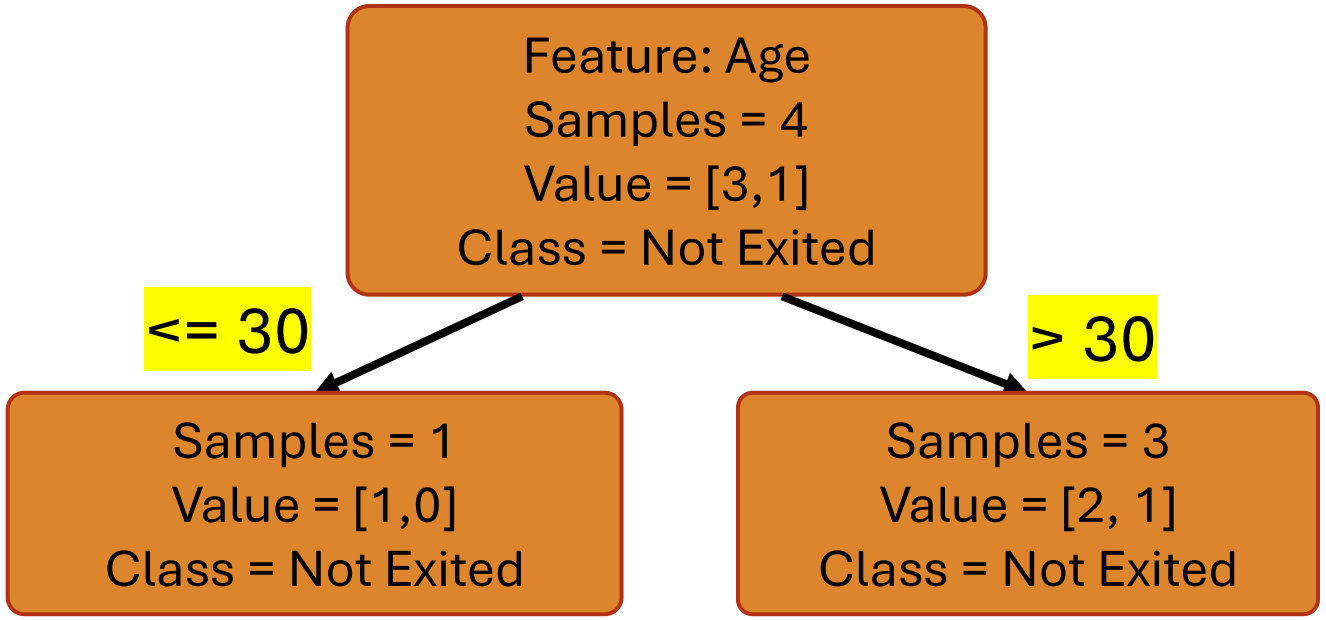
Choosing features and thresholds

- Suppose we are at node i
- Let n_i be the number of samples that reach node i
- We want to split these samples into a left branch and right branch to minimize error
  - Question: why two branches? Why not more?
  - Answer: you can do more branches, but doing so may cause **data fragmentation**
    - Too little data in each subtree, causing overfitting



Choosing features and thresholds

- Suppose we are at node i
- Let n_i be the number of samples that reach node i
- We want to split these samples into a left branch and right branch to minimize error
- Try all feature and threshold options, splitting on feature j_i with a threshold t_i
- Scalar feature example: try setting threshold to each sample value



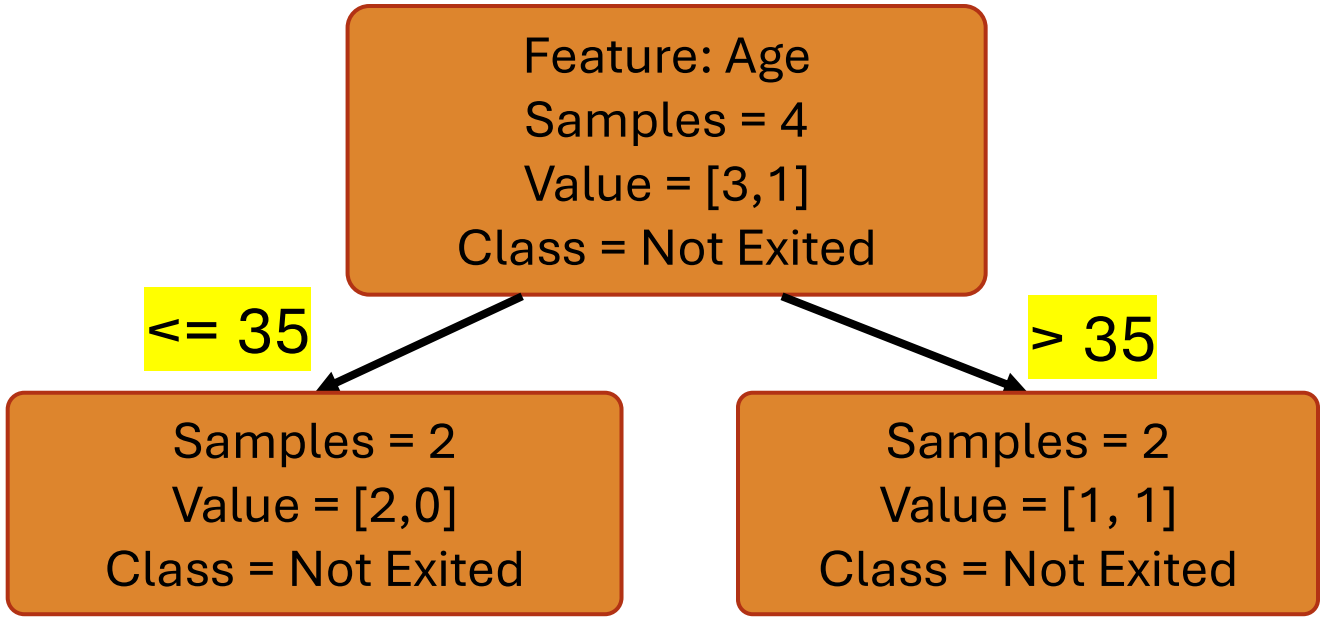
Samples:

[(Age 30, Not Exited), (Age 35, Not Exited), (Age 42, Exited), (Age 44, Not Exited)]

Try threshold = 30, 35, 42, and 44

Choosing features and thresholds

- Suppose we are at node i
- Let n_i be the number of samples that reach node i
- We want to split these samples into a left branch and right branch to minimize error
- Try all feature and threshold options, splitting on feature j_i with a threshold t_i
- Scalar feature example: try setting threshold to each sample value



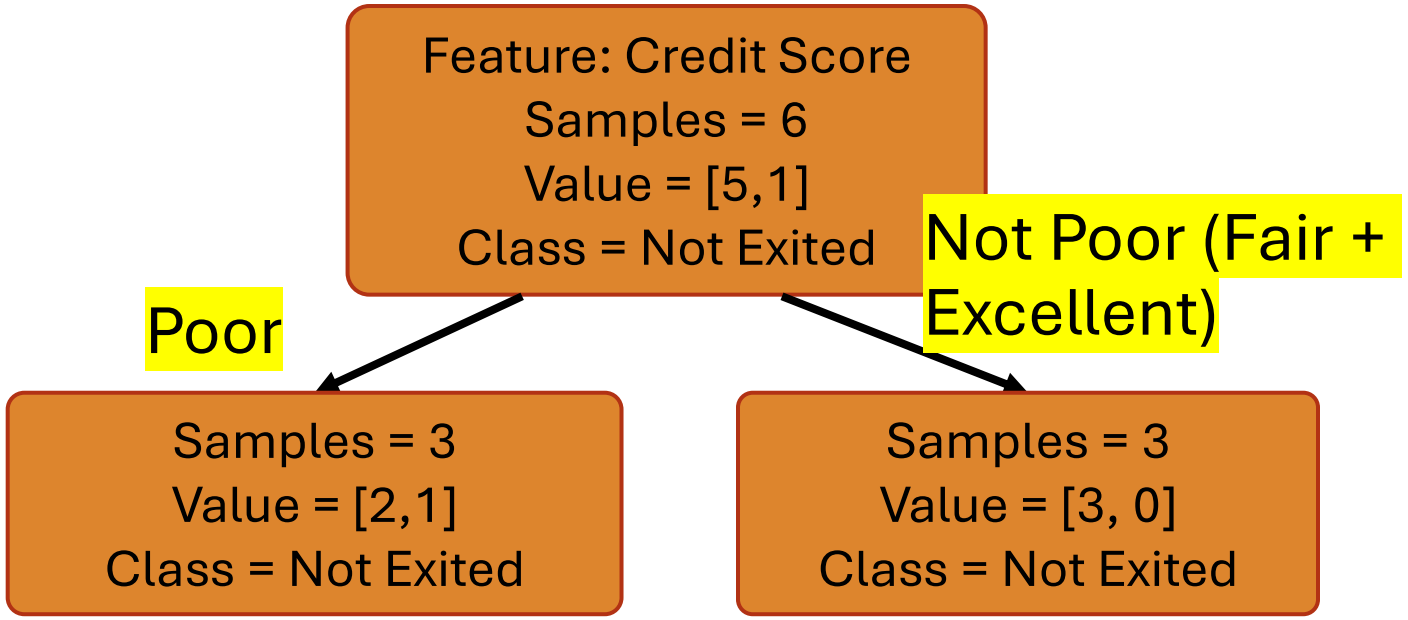
Samples:

[(Age 30, Not Exited), (Age 35, Not Exited), (Age 42, Exited), (Age 44, Not Exited)]

Try threshold = 30, 35, 42, and 44

Choosing features and thresholds

- Suppose we are at node i
- Let n_i be the number of samples that reach node i
- We want to split these samples into a left branch and right branch to minimize error
- Try all feature and threshold options, splitting on feature j_i with a threshold t_i
- Categorical feature example: try setting threshold to each category



Samples:

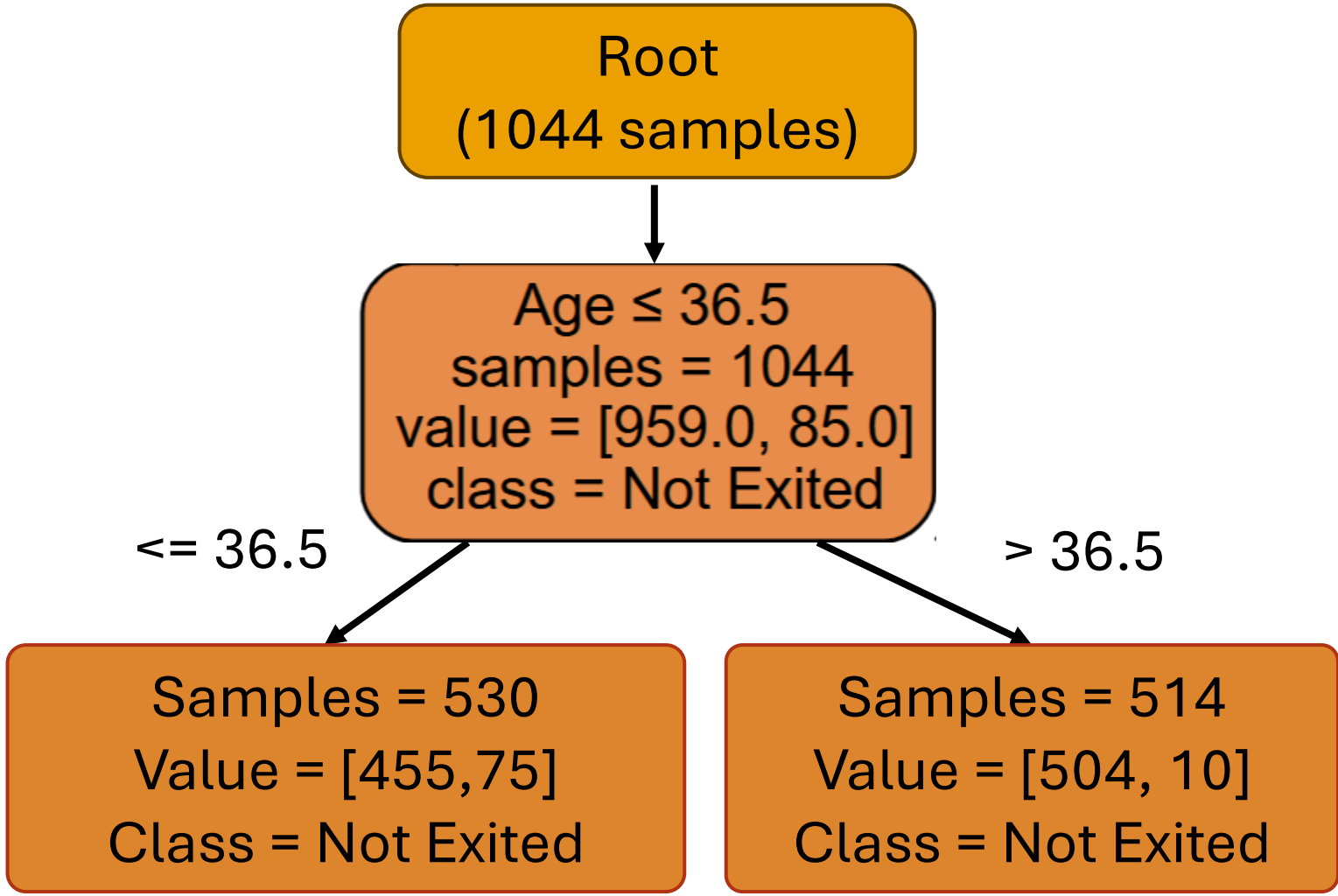
[(Poor, Exited), (Poor, Not Exited), (Poor, Not Exited), (Fair, Not Exited), (Fair, Not Exited), (Excellent, Not Exited)]

Try threshold = Poor, Fair, Excellent

Choosing features and thresholds

- Suppose we are at node i
- Let n_i be the number of samples that reach node i
- We want to split these samples into a left branch and right branch to minimize error
- Try all feature and threshold options, splitting on feature j_i with a threshold t_i
- Find the feature and threshold that minimizes:

$$\frac{n_{i_L}}{n_i} Cost_L(j_i, t_i) + \frac{n_{i_R}}{n_i} Cost_R(j_i, t_i)$$

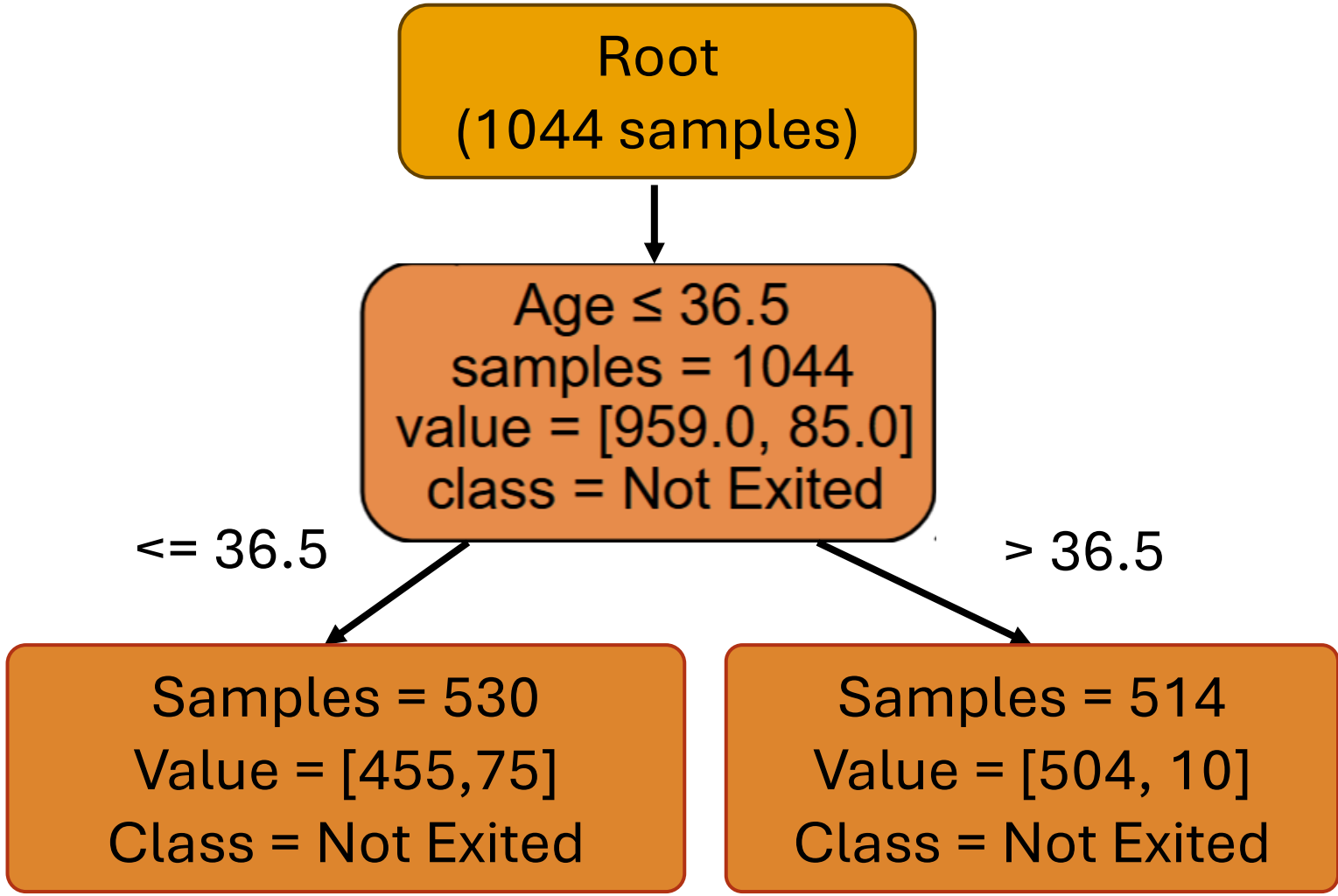


Choosing features and thresholds

- Find the feature and threshold that minimizes:

$$\frac{n_{i_L}}{n_i} Cost_L(j_i, t_i) + \frac{n_{i_R}}{n_i} Cost_R(j_i, t_i)$$

- n_i: # of samples at node i
- n_{i_L}: # of samples in left branch
- n_{i_R}: # of samples in right branch
- j_i: Feature chosen for split
- t_i: Threshold chosen for split
- $Cost_L, Cost_R$: cost of left and right branch, respectively
- What cost function to use?

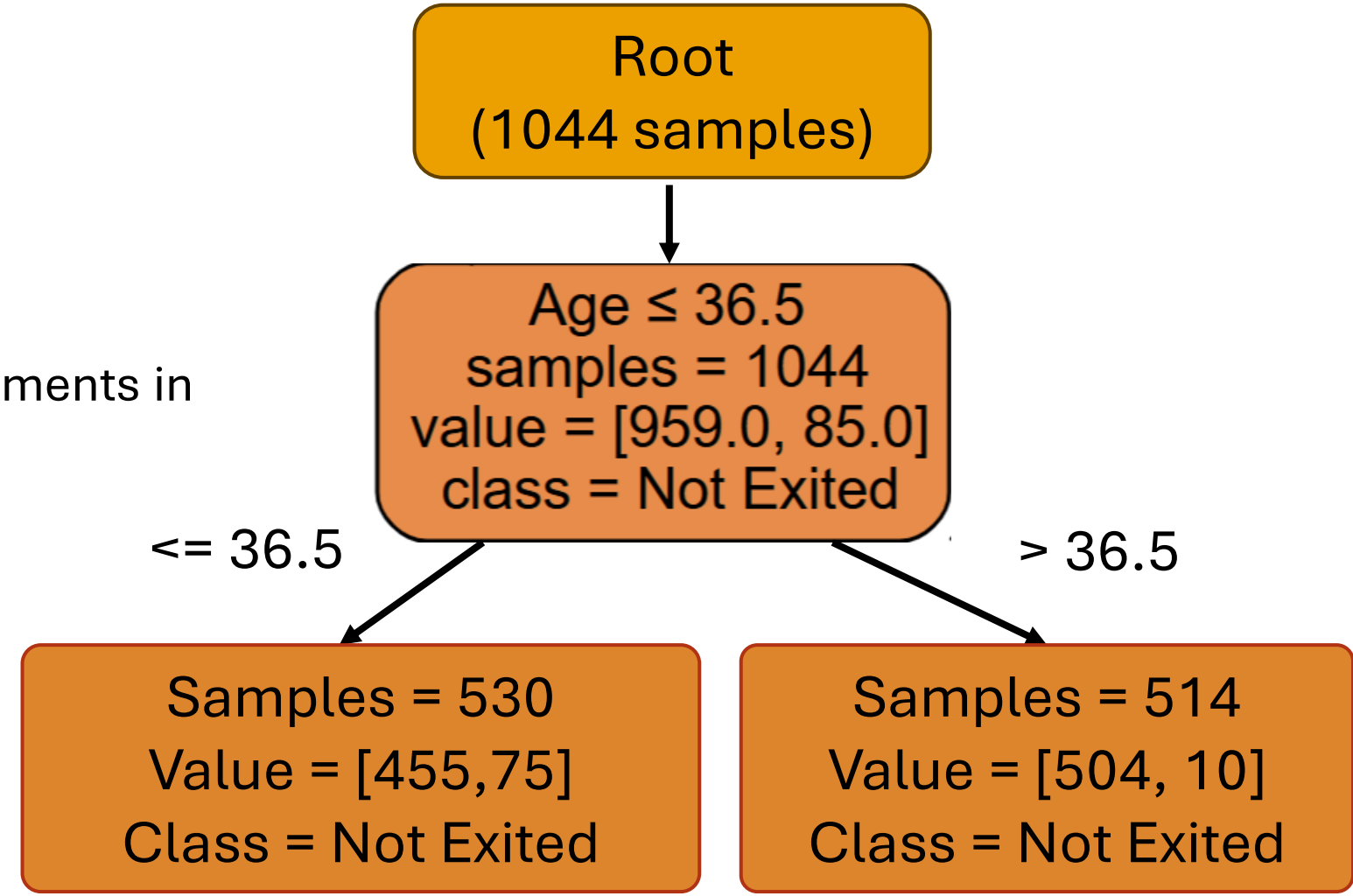


Cost Functions: Gini Impurity

- **Gini impurity (for k classes)**

$$1 - \sum_{c=1}^{k} P(class\ c)^2$$

- Minimum value?
  - 0, when the node is **pure**: all elements in a single class
  - $1 - 1^2 = 0$
- Maximum value?
  - $1 - \frac{1}{k}$, when classes have equal probability
  - $1 - \sum_{c=1}^{k} \left(\frac{1}{k}\right)^2 = 1 - \frac{k}{k^2} = 1 - \frac{1}{k}$



Decision Trees

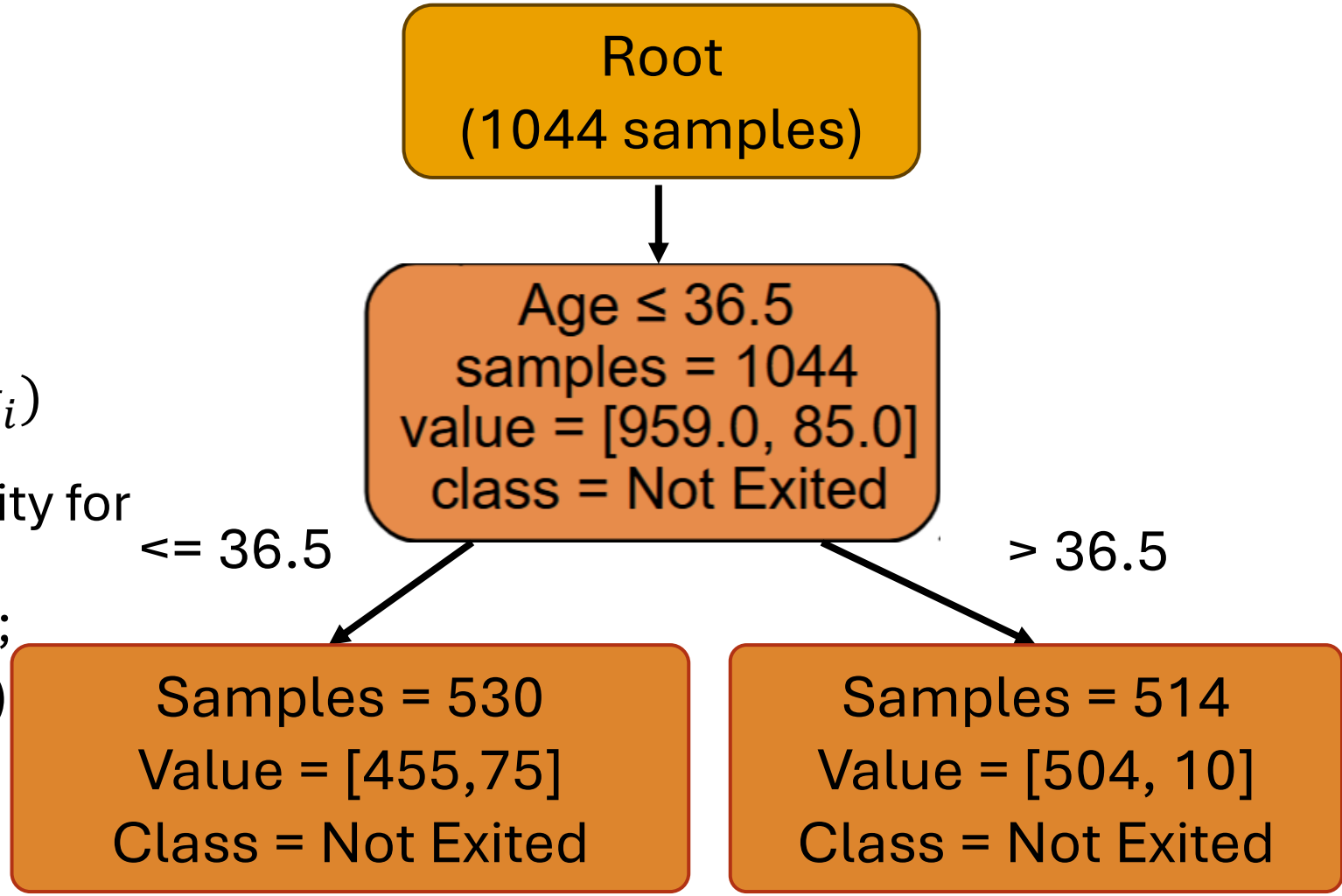
- **Gini impurity (for k classes)**

$$1 - \sum_{c=1}^{k} P(class\ c)^2$$

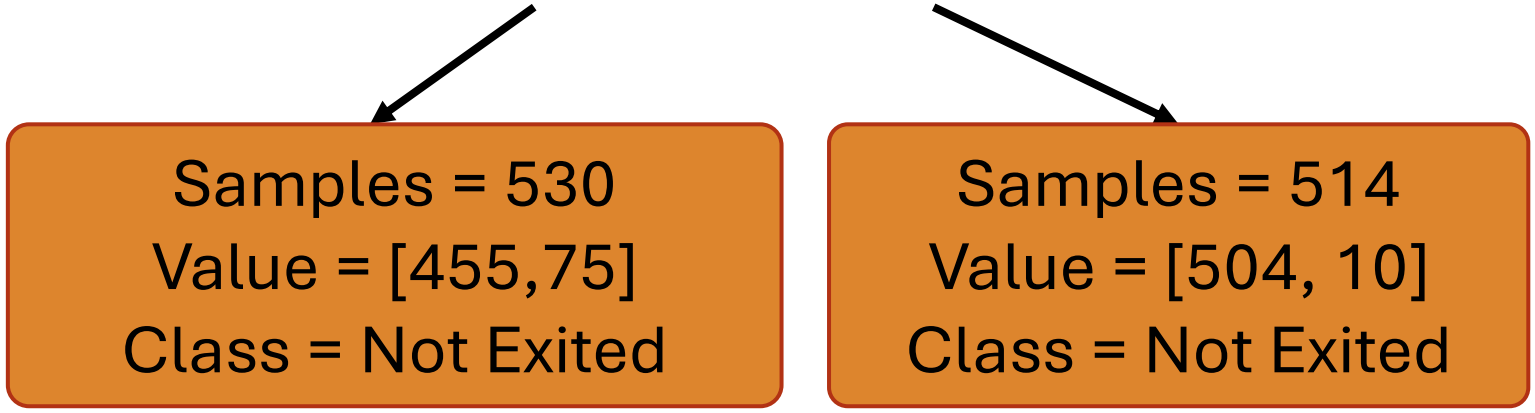
- Reminder: computing

$$\frac{n_{i_L}}{n_i} Cost_L(j_i, t_i) + \frac{n_{i_R}}{n_i} Cost_R(j_i, t_i)$$

- What is the cost using Gini Impurity for this node/split? (Can stop when formula is all numbers, no letters; no need to do the full calculation)



Decision Trees



- What is the cost using Gini Impurity for this node/split?

$$\frac{n_{i_L}}{n_i} Cost_L(j_i, t_i) + \frac{n_{i_R}}{n_i} Cost_R(j_i, t_i)$$

$$= \frac{n_{i_L}}{n_i}(1 - \sum_{c=1}^{k} P_L(class\ c)^2) + \frac{n_{i_R}}{n_i}(1 - \sum_{c=1}^{k} P_R(class\ c)^2)$$

$$= \frac{530}{1044}(1 - (P_L(exited)^2 + P_L(not\ exited)^2)) + \frac{504}{1044}(1 - (P_R(exited)^2 + P_R(not\ exited)^2))$$

$$= \frac{530}{1044}(1 - (\frac{75}{530}^2 + \frac{455}{530}^2)) + \frac{504}{1044}(1 - (\frac{10}{514}^2 + \frac{504}{514}^2))$$

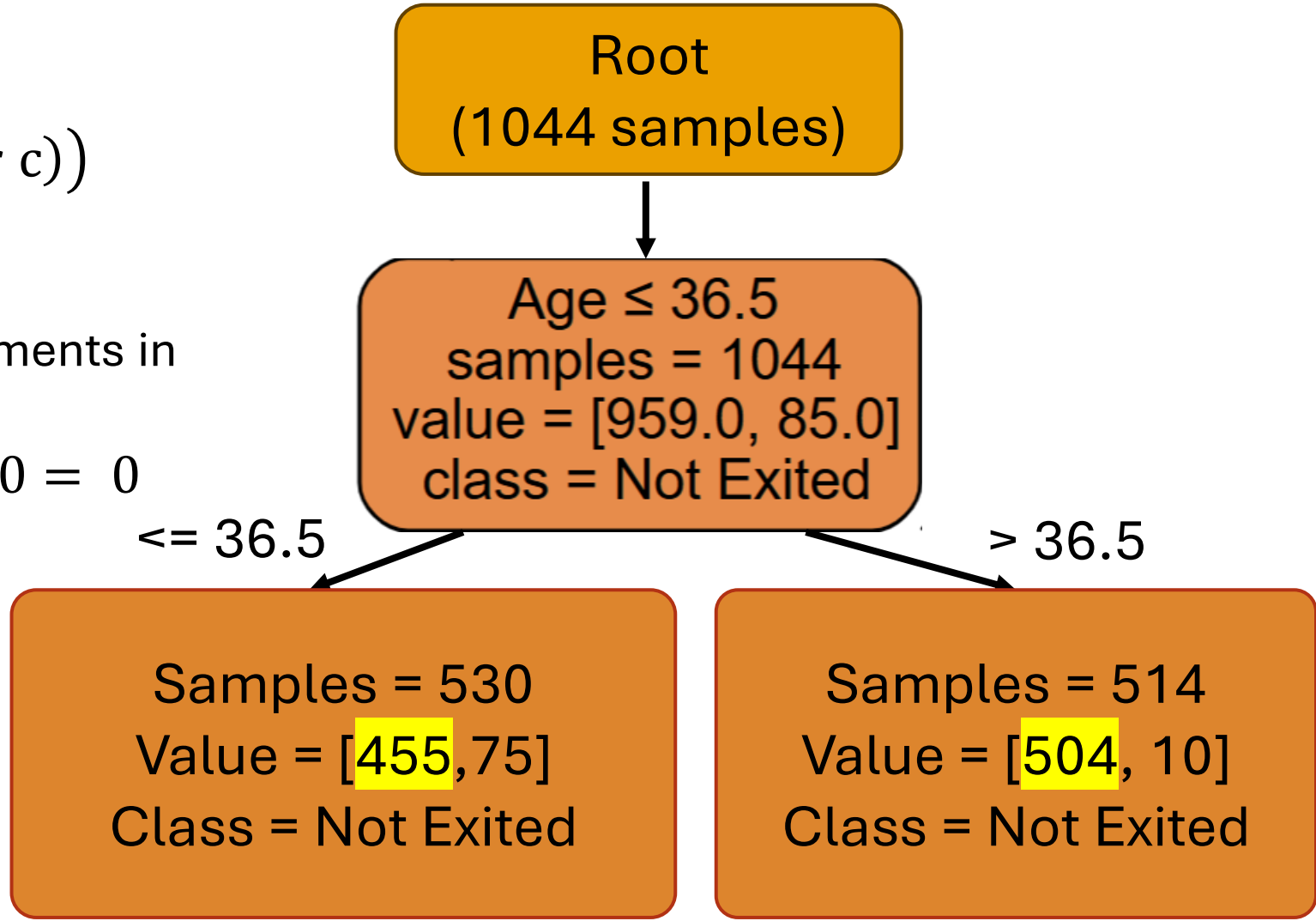
$$= 0.12 + 0.02 = 0.14$$

Decision Trees

- **Entropy (for k classes)**

$$-\sum_{c=1}^{k} P(class\ c) * log_2\big(P(class\ c)\big)$$

- Minimum value?
  - 0, when the node is **pure**: all elements in a single class
  - $-\sum_{c=1}^{k} 1 * log_2(1) = -\sum_{c=1}^{k} 1 * 0 = 0$
- Maximum value?
  - $-\sum_{c=1}^{k} \frac{1}{k} * log_2\left(\frac{1}{k}\right)$
  - $= -\sum_{c=1}^{k} \frac{1}{k} * -log_2(k)$
  - $= log_2(k)$ when classes have equal probability

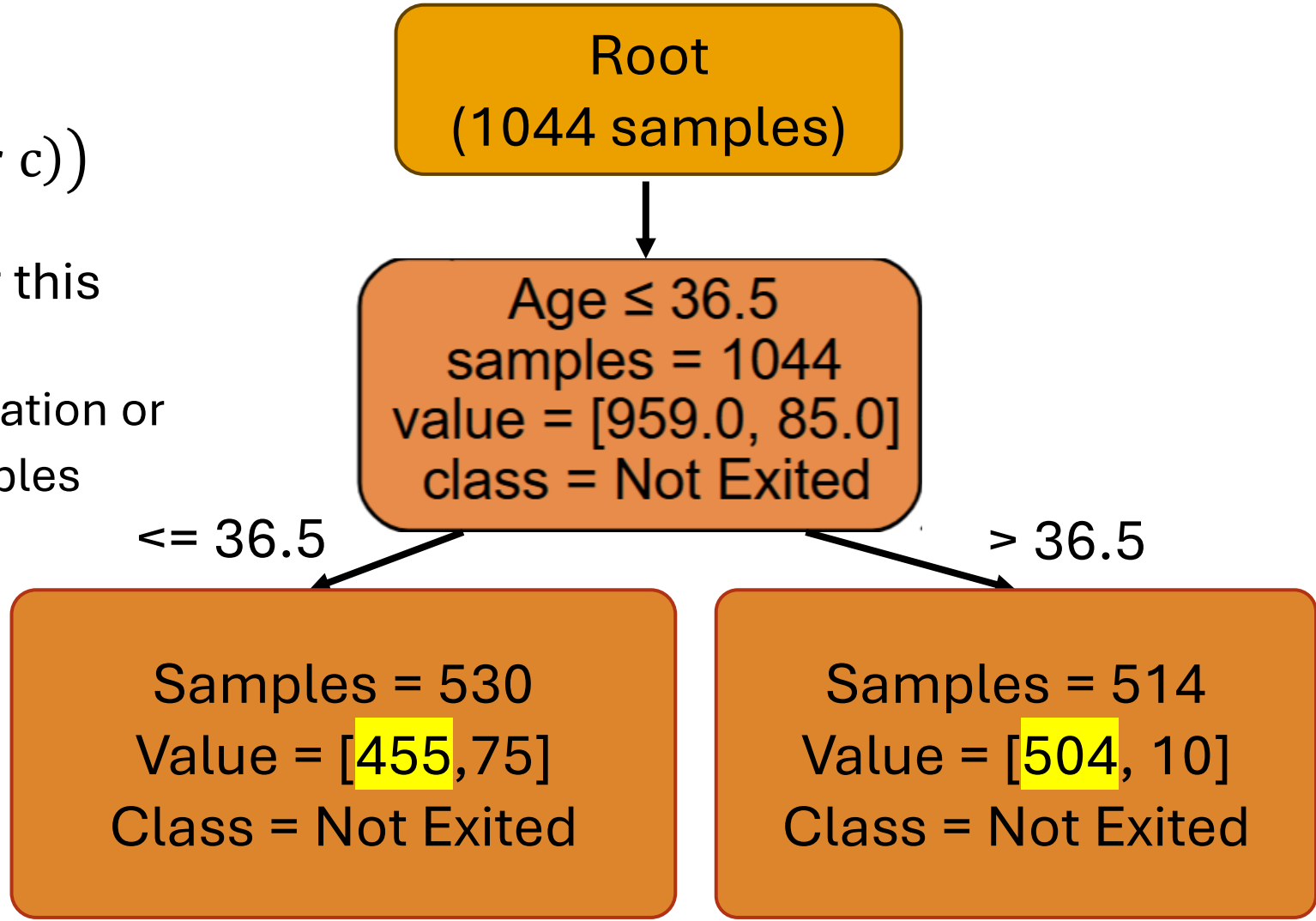


Decision Trees

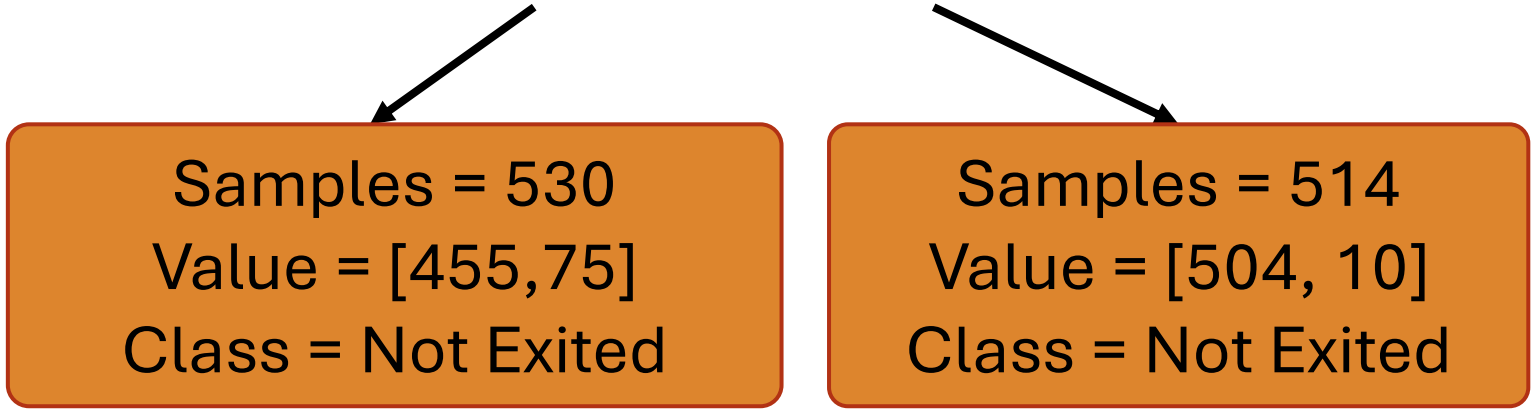
- **Entropy (for k classes) cont'd**

$$-\sum_{c=1}^{k} P(class\ c) * log_2\big(P(class\ c)\big)$$

- What is the cost using entropy for this split?
  - Again, no need to do full computation or logs: just plug in values for variables



Decision Trees



- What is the cost using entropy for this node/split?

Left node only for time

$$\frac{n_{i_L}}{n_i} Cost_L(j_i, t_i)$$

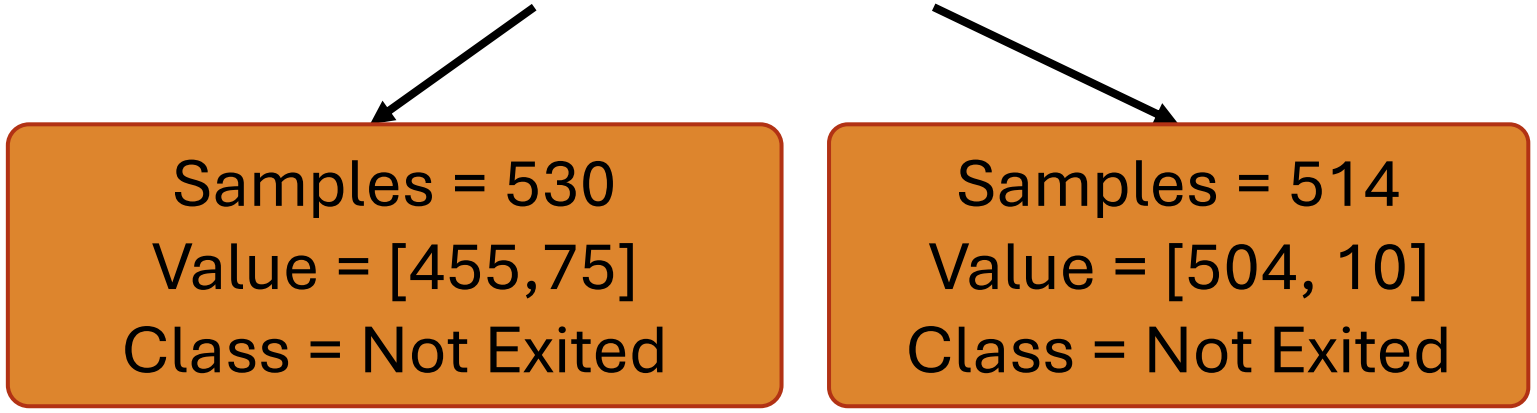
$$= \frac{n_{i_L}}{n_i} (- \sum_{c=1}^{k} P_L(class\ c) * log_2(P_L(class\ c)))$$

$$= \frac{530}{1044} (-(P_L(exited) * log_2(P_L(exited)) - (P_L(\text{not } exited) * log_2(P_L(\text{not } exited)))$$

$$= \frac{530}{1044} (-\frac{75}{530} * log_2(\frac{75}{530}) - \frac{455}{530} * log_2(\frac{455}{530}))$$

$= 0.11$ (repeat for right node and add together - total cost is 0.18)

Decision Trees



- Comparing costs
  - Gini impurity cost: 0.14
  - Entropy cost: 0.18
  - Gini impurity maximum: 1 – ½ = 1 – 0.5 = 0.5
  - Entropy maximum: $log_2(2) = 1$

- May also hear the term **information gain**
  - This is the reduction in **entropy** from the parent to the child nodes
  - Formula: E_{parent}- $\frac{n_{i_L}}{n_i} E_{\mathrm{L}} + \frac{n_{i_R}}{n_i} E_{\mathrm{R}}$, where $E_{parent}, E_{\mathrm{L}}, E_{\mathrm{R}}$ is the entropy of the parent, left, and right nodes

Algorithm: greedy, recursive

- Algorithm:

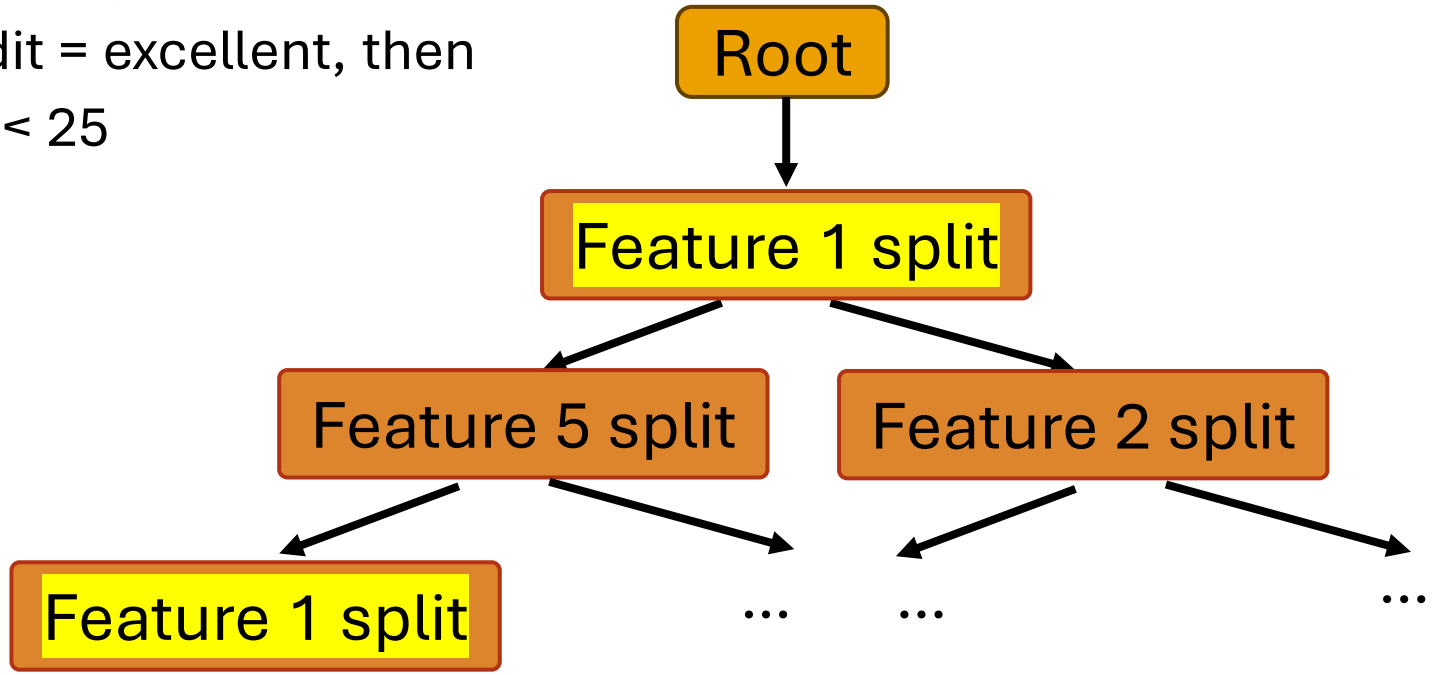
```
BuildTree(node)
  If termination criterion is met:
    Stop
  Else:
    Split(node)
    For child in node:
      BuildTree(child)
```

Greedy choice: best action in the moment

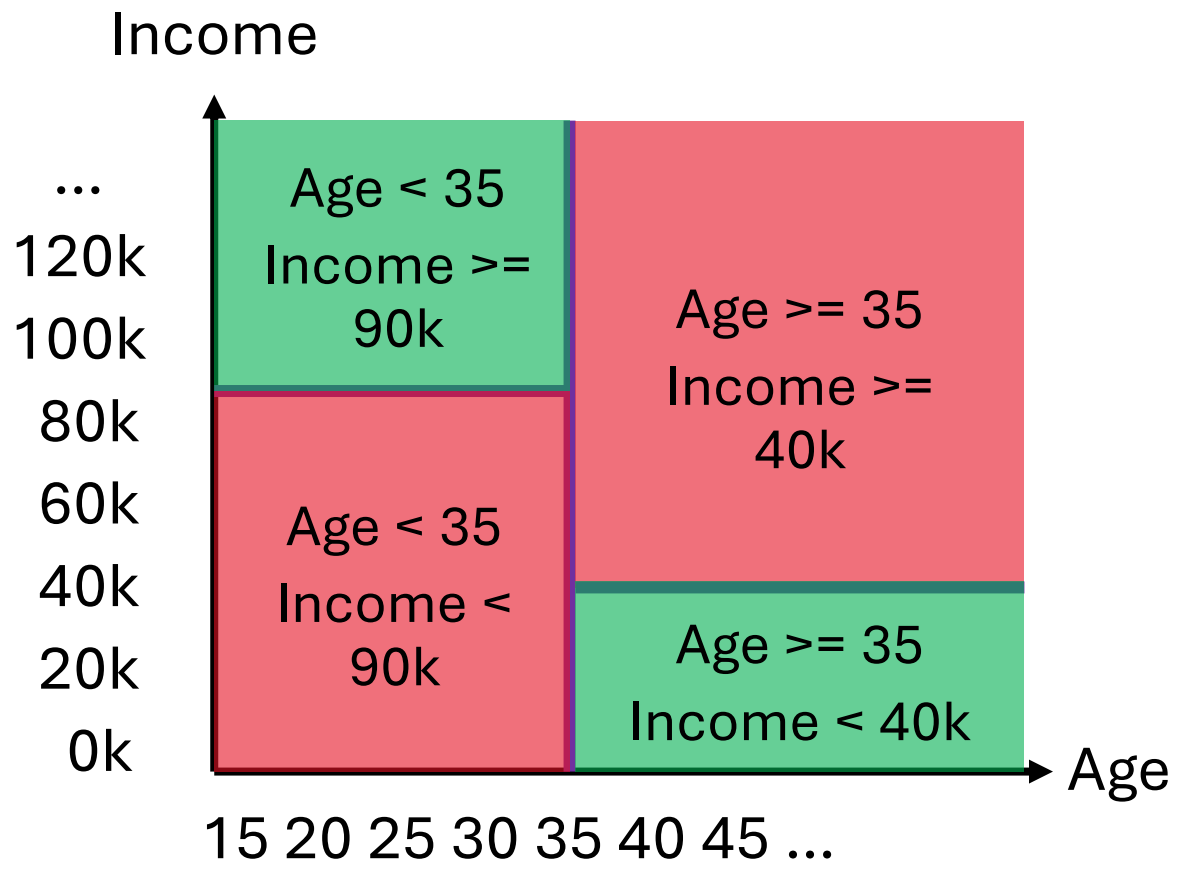
Recursive: a function that calls itself

Decision Trees

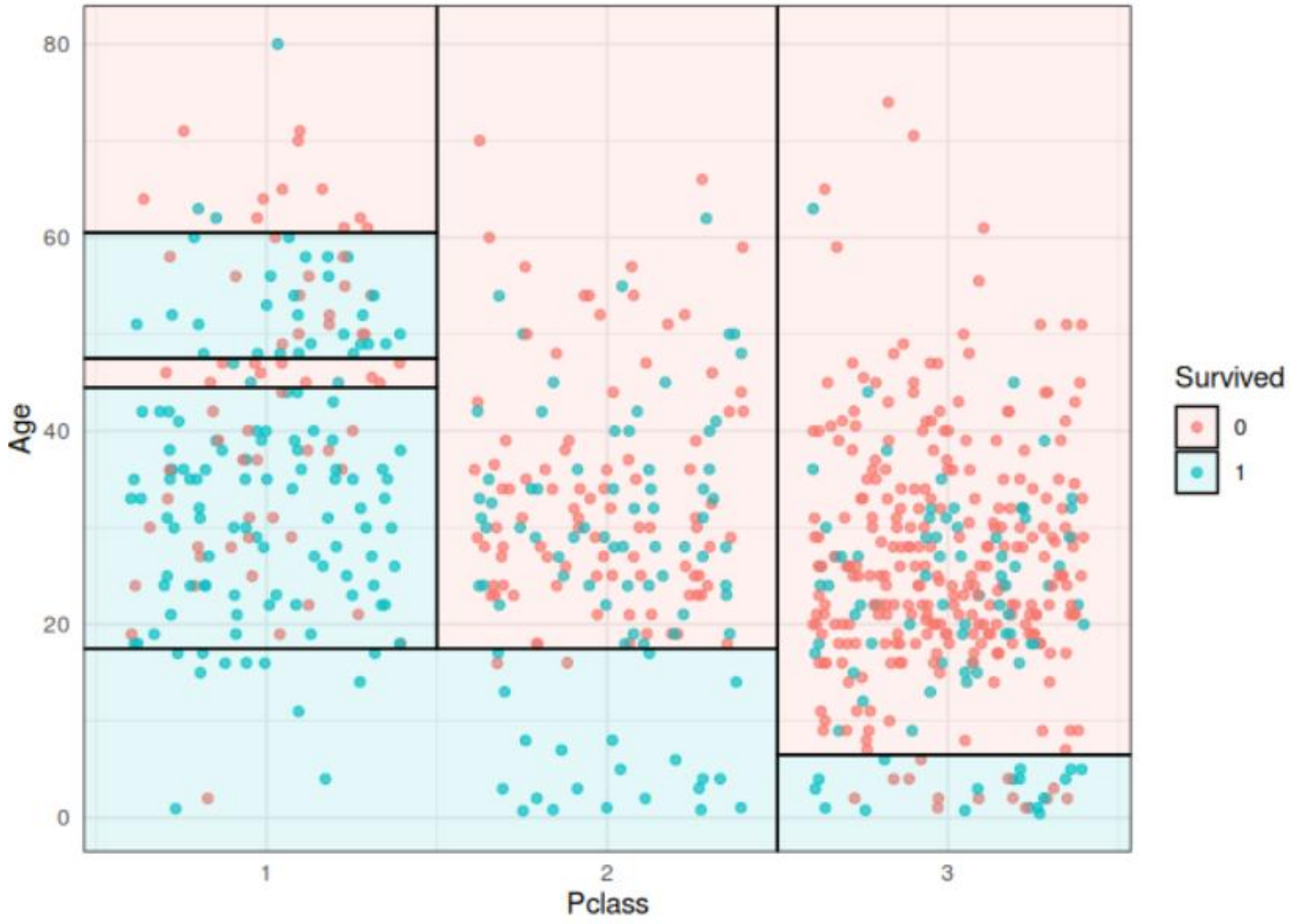
- Decision trees can have multiple splits per feature
- Example
  - Split on age < 35, then
  - Split on credit = excellent, then
  - Split on age < 25



Splits partition the space in rectangles

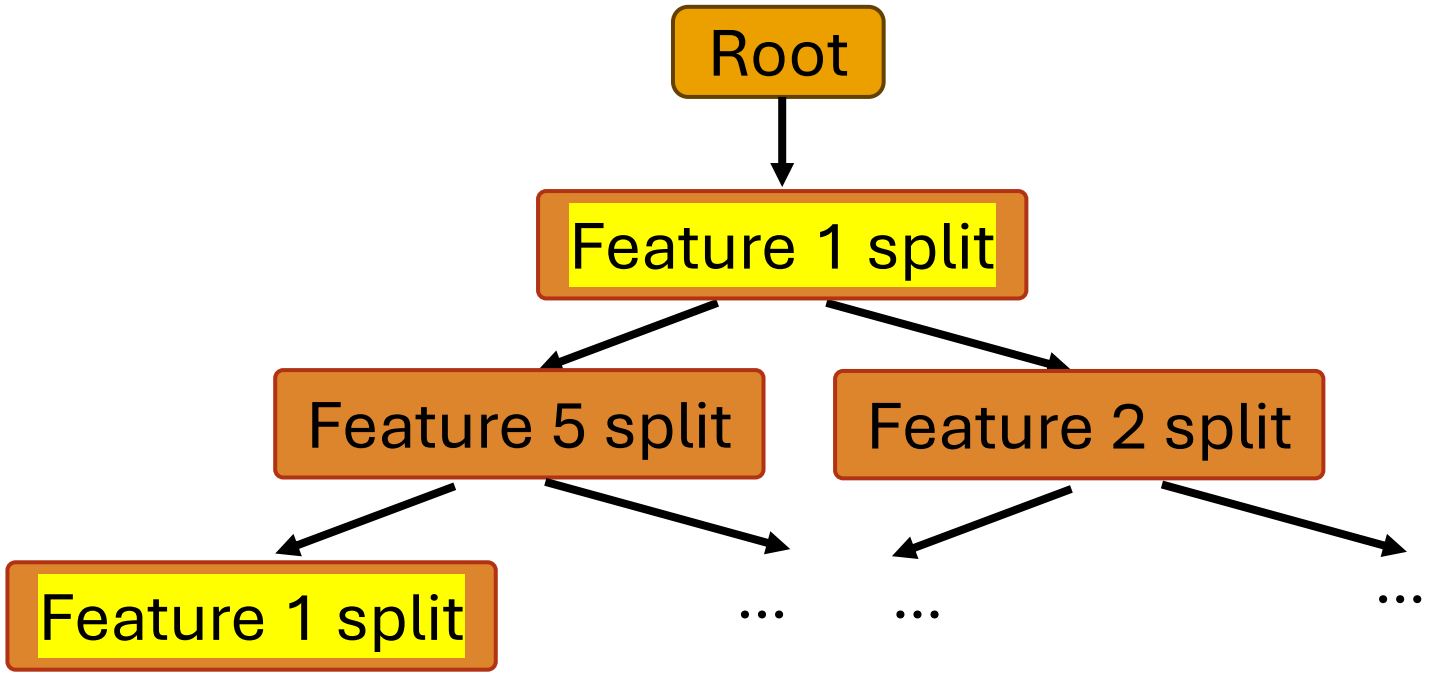


Splits can get complex



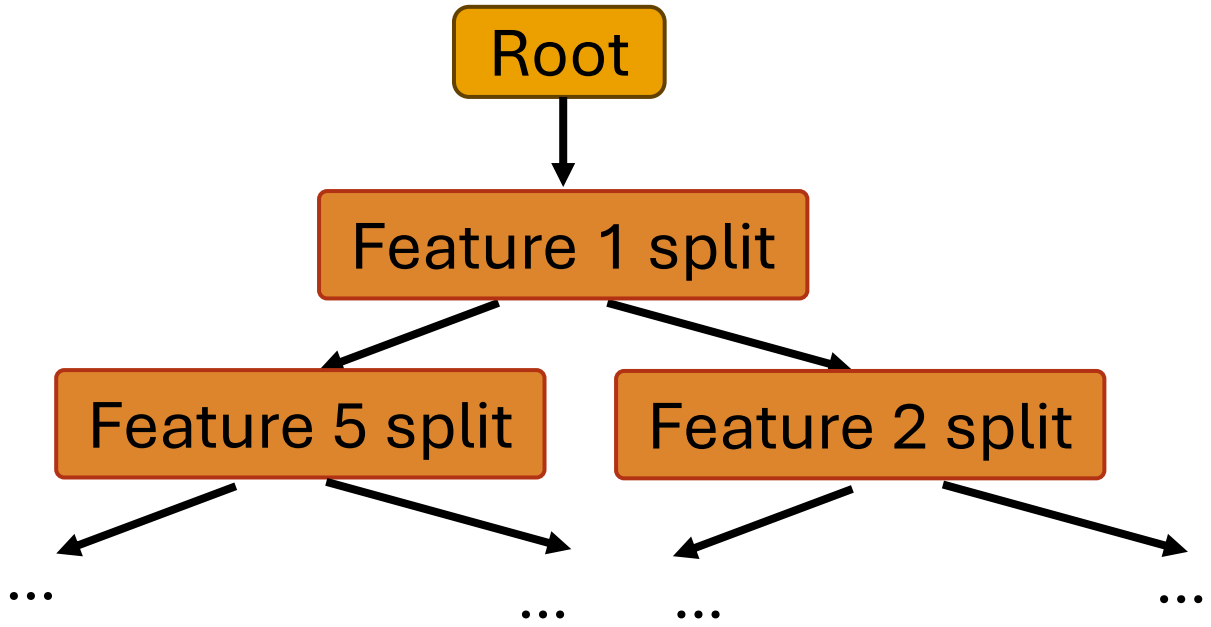
Termination criteria

- Decision trees split until nodes are **pure** or until they've reached some other termination criterion



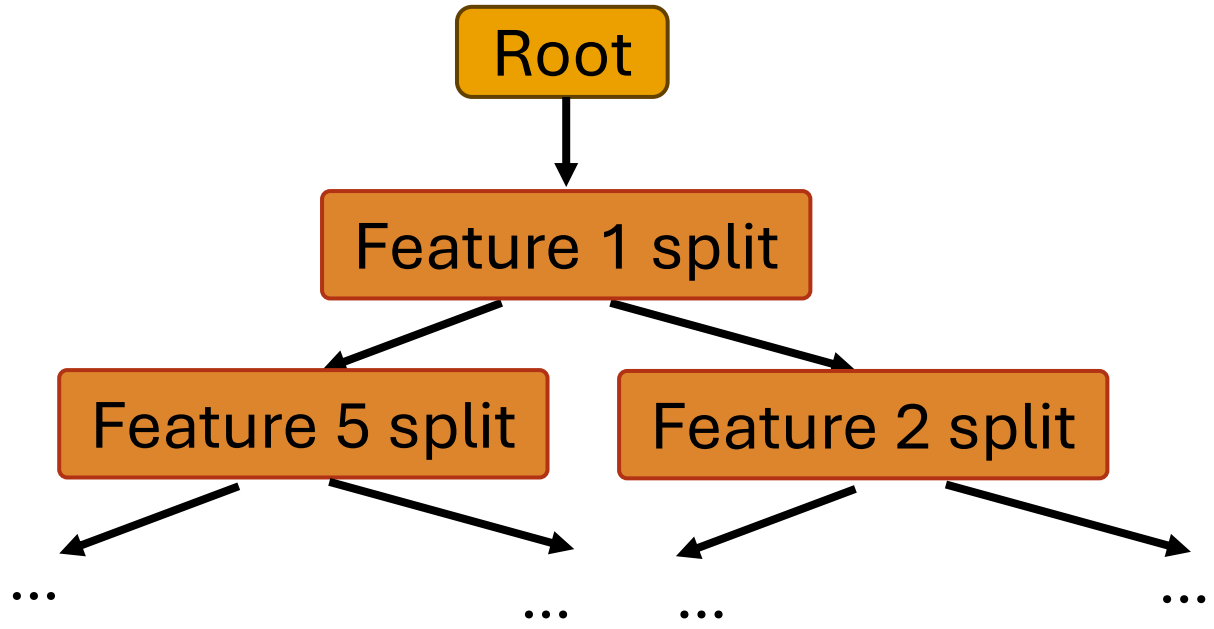
Decision Trees

- If you let a decision tree split until all leaves are pure (assuming no label noise), what would happen to our training error?



Decision Trees

- If you let a decision tree split until all leaves are pure (assuming no label noise), what would happen to our training error?
- The tree will achieve 0 error on the training set
- May overfit!



Decision Trees and Overfitting

- Decision trees are prone to overfitting
  - Deep trees are more likely to overfit data

- **Bias and variance**
  - Decision trees tend to have low bias (with enough depth) and high variance

- Small changes in datasets can lead to big classification changes

- In Logistic Regression and other parametric methods we discussed, large parameters could indicate overfitting. Discuss: what about decision trees?
  - Possible numbers to consider: depth, training error, number of samples at leaf nodes

Decision Trees and Overfitting

- Decision trees that have overfit will often have low training error, high depth, and a small number of samples at each leaf node
  - Example: every leaf node could have 1 or 2 samples, and just choose a split that perfectly classifies
  - Probably not very helpful: like saying the customer might churn if they are exactly 34 years old, have fair credit scores, have exactly 4 products with the bank, and a bunch of other features.
- We can create new **termination criteria**
  - Set maximum depth
  - Set minimum number of samples at each node
  - Stop if split doesn't decrease error/increase information gain by more than some set amount (5%, etc)
- Or, we can train trees to its maximum depth (no more possible splits), and then **prune** by merging subtrees back together

Pros and Cons of Decision Trees

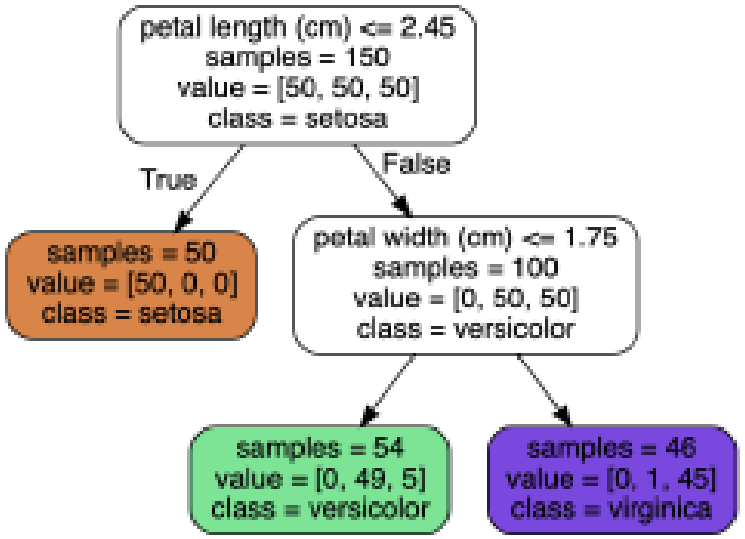
- Pros
  - The decision boundaries are usually **interpretable** (we can see exactly why a decision was made), although a very deep tree may be more confusing
  - They can handle mixed discrete and continuous inputs
  - No need to standardize data
  - Automatically select variables (no need for LASSO)
  - Easy multi-class classification
  - Can create non-linear decision boundaries (although they must be parallel to axes/rectangular)
  - They fit quickly, and can handle missing input features
    - One way is to create a new value, “missing”, for categorical data
    - Another is **surrogate splits**: Find “backup” variables (highly correlated features) that find similar partitions to the missing variable

Pros and Cons of Decision Trees

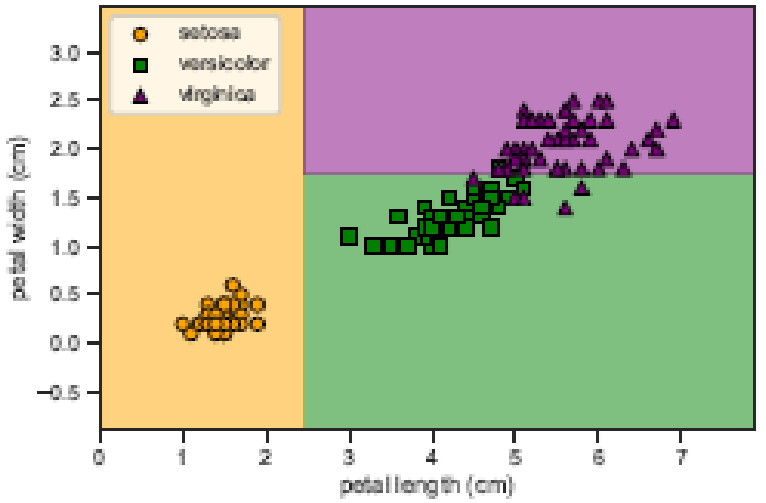
- Cons
  - Greedy, so prediction is less accurate than some other models
  - Not great for many related continuous features, e.g. pixels
  - **Unstable**: small changes in input data can have large effects on tree structure

Instability

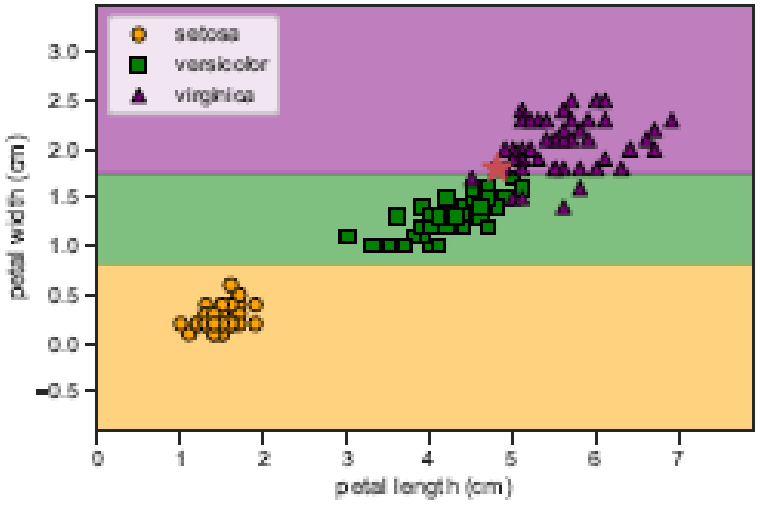
- One data point omitted (image from Probabilistic Machine Learning textbook)



(a)



(b)



(c)

Break

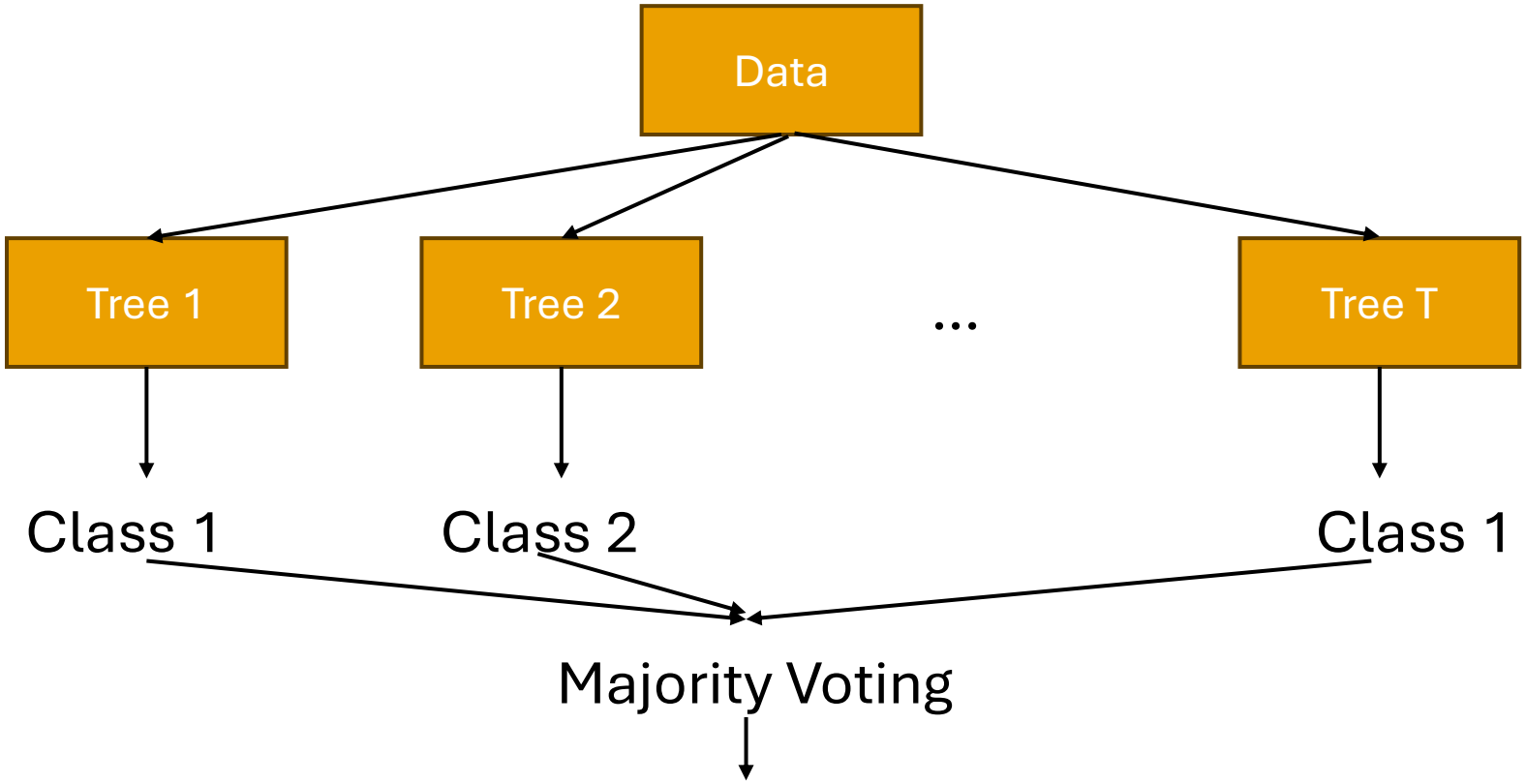
Get some water, stretch your legs, chat, be back in 10

Ensemble Methods

- Decision trees have a lot of pros!
- But also some cons that can make them less optimal for some data
- Can we improve trees to lesson some of these cons?
- A **model ensemble** is a collection of models (often **weak**; slightly better than random guessing)
- We can use model ensembles to combine trees!
- We'll cover
  - **Random Forest** (Bagging model ensemble)
  - **AdaBoost** (Boosting model ensemble)

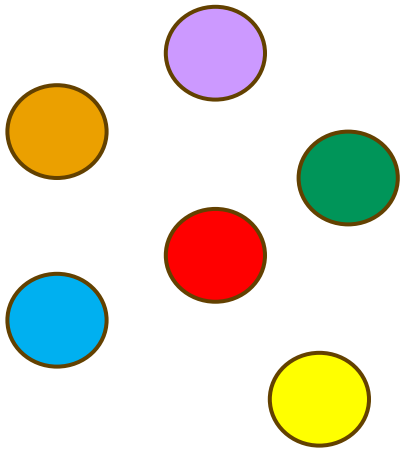
Random Forests

- A collection of T decision trees
- Each decision trees “votes” for a prediction
- Ensemble predicts the majority vote over all trees

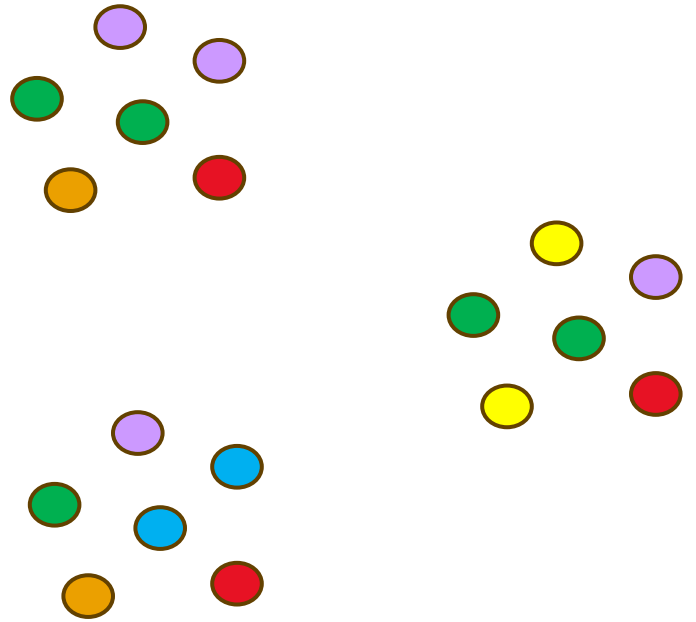


Random Forests

- If we have one dataset, how are the T trees different?
- We can create many (very similar, but different) datasets by randomly sampling from the original dataset **with replacement**
  - Meaning samples can be selected multiple times



Original Dataset (size n)



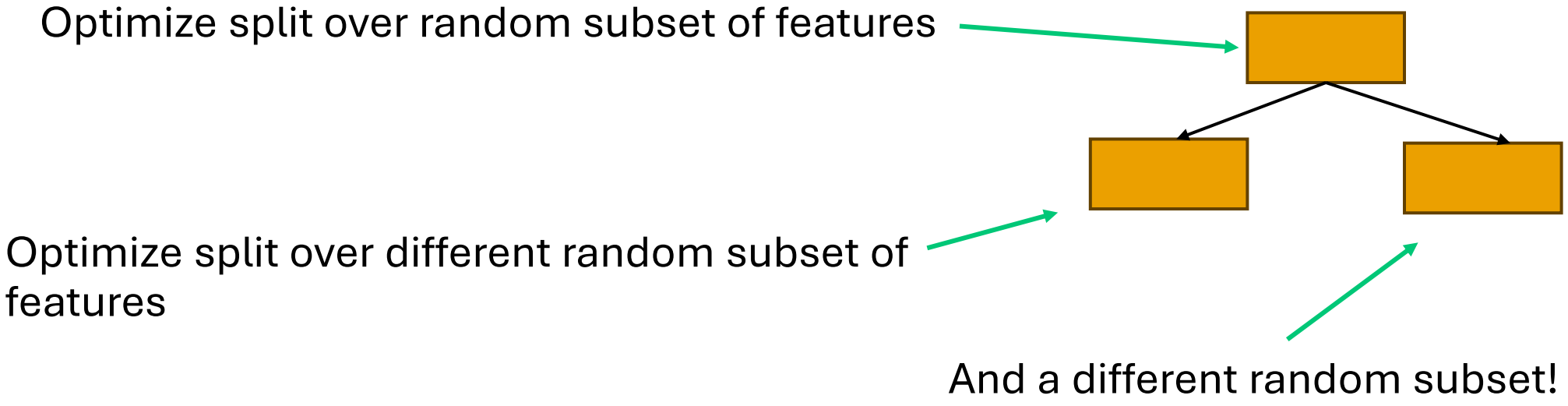
New Datasets (each size n)

Random Forests

- This technique is called **bootstrapping,** or **bootstrap aggregating**
- Disadvantage: each base model sees on average 63% of data
  - The probability of a single item not being selected from a set of size n in n draws is$\left(1-\frac{1}{n}\right)^{n}$
  - As n goes to infinity, this equation becomes $e^{-1} \approx 0.37$, so 1-0.37 = 0.63 of datapoints will be selected
- Advantage:
  - Prevents the ensemble of trees from overly relying on any individual training sample
  - **Out of Bag (OOB)** error: can estimate performance using training data by only using trees not trained on each point
- The T trees will be quite different despite similar training data because decision trees are **unstable**!

Random Forests

- Additionally, at every node of each tree, a random subset of features is selected for use
- Random forests optimize over a subset of feature splits, not every possible feature split
- For classification, it is recommended to start with using $\sqrt{D}$ features



Random Forests

- Question: since we want very unstable trees, should they be deep or shallow?

Random Forests

- Question: since we want very unstable trees, should they be deep or shallow?
- Answer: very deep! We want them to overfit

- Overfitting gives us trees with high variance and low bias
  - If you “average” over many high variance models, you end up with a model with low bias
  - If we average over many overfit/high variance trees, we get an ensemble with low bias and a lower variance (as long as there are plenty of trees)

Random Forest Application

- Random Forests were used in the Microsoft Kinect system to identify human poses (3D positions of body joints) from depth camera images

Real-Time Human Pose Recognition in Parts from Single Depth Images

Jamie Shotton Andrew Fitzgibbon Mat Cook Toby Sharp Mark Finocchio
Richard Moore Alex Kipman Andrew Blake
Microsoft Research Cambridge & Xbox Incubation

Abstract

We propose a new method to quickly and accurately predict 3D positions of body joints from a single depth image, using no temporal information. We take an object recognition approach, designing an intermediate body parts representation that maps the difficult pose estimation problem into a simpler per-pixel classification problem. Our large and highly varied training dataset allows the classifier to estimate body parts invariant to pose, body shape, clothing, etc. *Finally we generate confidence-scored 3D proposals of several body joints by reprojecting the classification result and finding local modes.*

The system runs at 200 frames per second on consumer hardware. Our evaluation shows high accuracy on both synthetic and real test sets, and investigates the effect of several training parameters. We achieve state of the art accuracy in our comparison with related work and demonstrate improved generalization over exact whole-skeleton nearest neighbor matching.

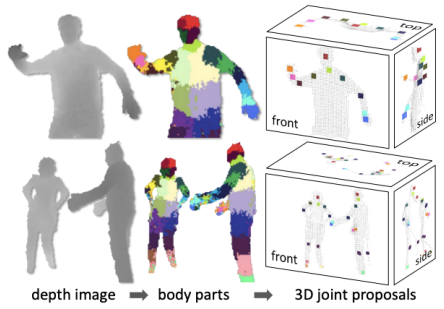


Figure 1. **Overview.** From an single input depth image, a per-pixel body part distribution is inferred. (Colors indicate the most likely part labels at each pixel, and correspond in the joint proposals). Local modes of this signal are estimated to give high-quality proposals for the 3D locations of body joints, even for multiple users.

joints of interest. Reprojecting the inferred parts into world

Random Forest Pros and Cons

- Pros
  - Tends to require less hyperparameter tuning than other methods
    - More trees is always better (but takes longer)
    - Other hyperparameters have fairly small affect on performance
  - Efficient: trees can be learned in parallel
    - If you have multiple cores/computers to train on, you could train a single tree on each one simultaneously
  - Can classify non-linear data (as long as the boundaries can be composed of axis-aligned rectangles)
- Cons
  - Computationally expensive
  - Less interpretable than a single tree

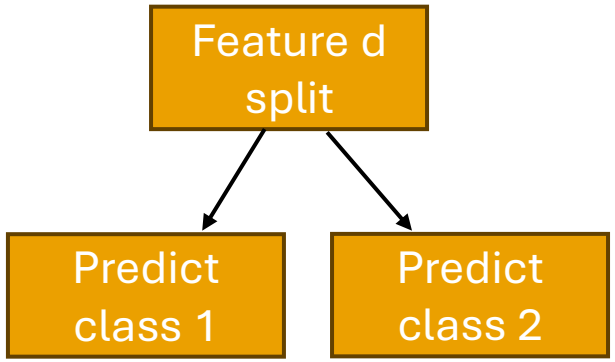
AdaBoost

- AdaBoost is also made of many decision trees, but uses **boosting** instead of **bagging**
- Tree depth
  - For Random Forests, we train high depth trees that overfit
  - For AdaBoost, we only train **decision stumps**
    - Reminder: stumps are decision trees with only 1 level (so one feature split)
- Final decision
  - For Random Forests, we use majority voting
  - For AdaBoost, we take a **weighted majority vote**
- Dataset
  - For Random Forests, we use random sampling with replacement and random subsets of features
  - For AdaBoost, we use the whole dataset (and all features)with each datapoint assigned a weight

AdaBoost Tree Depth

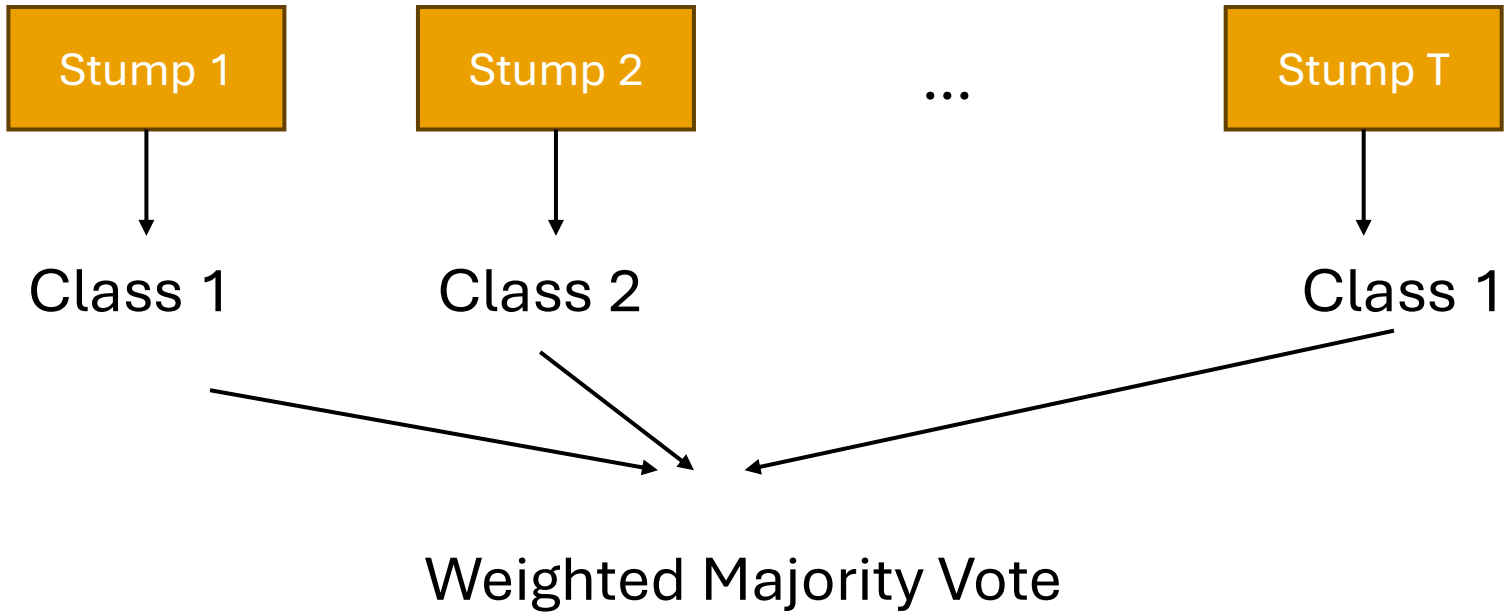
- Tree depth
  - For Random Forests, we train high depth trees that overfit
  - For AdaBoost, we only train **decision stumps**
    - Reminder: stumps are decision trees with only 1 level (so one feature split)

Stump Example



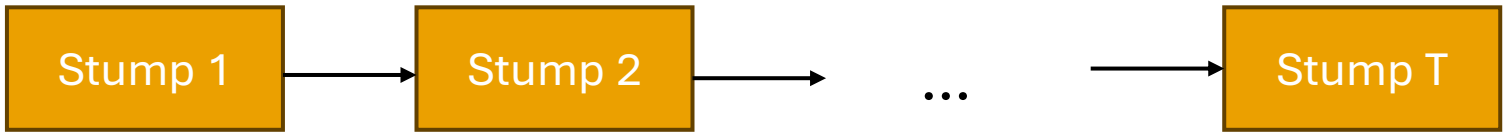
AdaBoost Final Decision

- Final decision
  - For Random Forests, we use majority voting
  - For AdaBoost, we take a **weighted majority vote**
  - Note: AdaBoost does **binary** classification (-1, +1)
  - $\hat{y} = sign\left(\sum_{t=1}^{T} \widehat{w_t}\, \widehat{f_t}(x)\right)$
  - w_t: weight of stump t
  - $\widehat{f_t}(x)$: decision of stump t



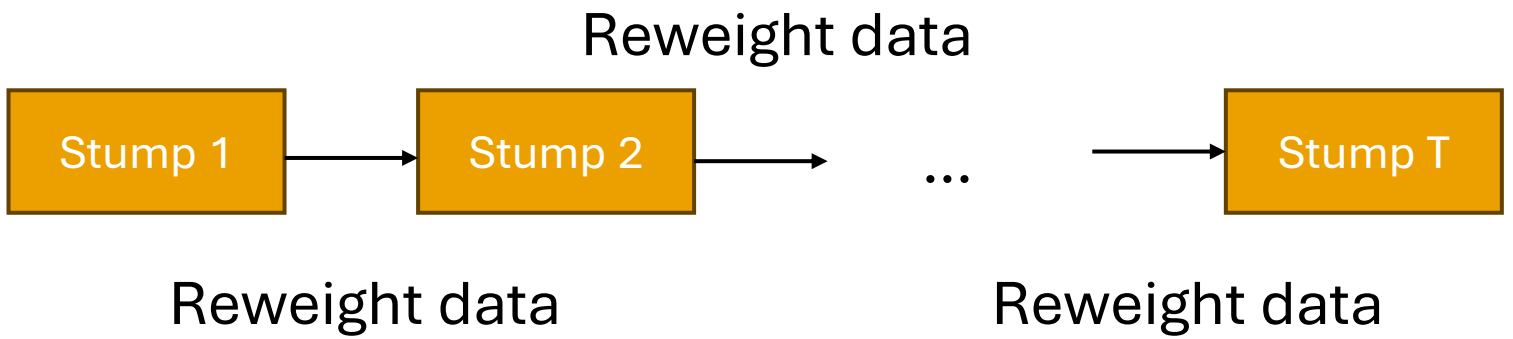
Training AdaBoost

- Instead of training all trees at once, we train trees in succession
  - Use the outcome of each stump to reweight datapoint and affect learning
- We have two types of weights
  - High-weight datapoints (weighted with α_i are those frequently misclassified by **previous trees**
  - $\hat{y} = sign\left(\sum_{t=1}^{T} \widehat{w_t}\, \widehat{f_t}(x)\right)$ where w_t: weight of stump t
- Intuition behind weights
  - w_t should be higher for **accurate** stumps
  - α_i should be higher for datapoints that are difficult to classify



AdaBoost Dataset

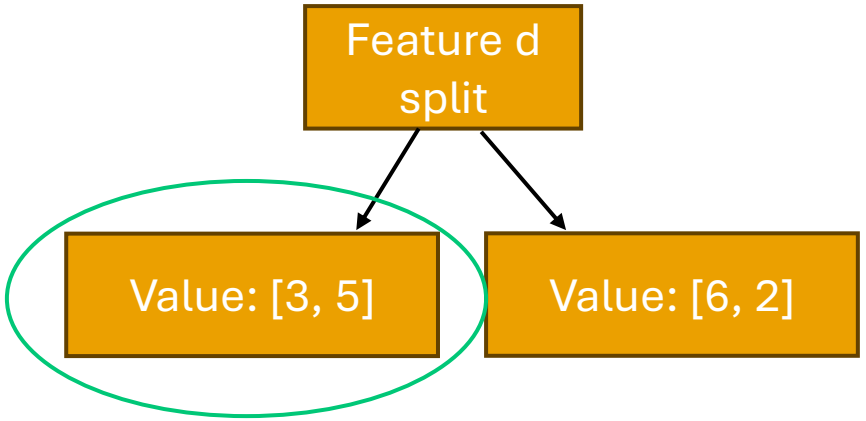
- Dataset
  - For Random Forests, we use random sampling with replacement and random subsets of features
  - For AdaBoost, we use the whole dataset (and all features) with each datapoint assigned a weight
- High-weight datapoints are those frequently misclassified by **previous trees**

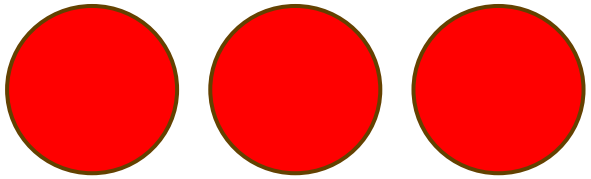


Training Trees with Weighted Data

- Leaf node predictions also change

- Instead of just finding the majority, we find the **weighted majority**





Three class 1 datapoints with weight 2: **3*2 = 6**

Predict class 1

Five class 2 datapoints with weight 0.5: **5*0.5 = 2.5**

Training Trees with Weighted Data

- Now that datapoints have weights, we do not just want to minimize Gini Impurity or entropy
  - We need to account for weights!
- We want to minimize **weighted classification error**

$$WeightedError(f_t) = \frac{\sum_{i=1}^{n} \alpha_{i,t} \mathbf{1}\{\hat{f}_t(x_i) \neq y_i\}}{\sum_{i=1}^{n} \alpha_{i,t}}$$

- In words, this is the sum total weight of all mistaken predictions over the sum total weight of all predictions
- Now, higher weighted points contribute more to the model error!

AdaBoost: Model Weights

- Intuition behind weights
  - w_t should be higher for **accurate** stumps
- AdaBoost uses the formula $\widehat{w}_t = \frac{1}{2}\ln\left(\frac{1-WeightedError(\hat{f}_t)}{WeightedError(\hat{f}_t)}\right)$

- $WeightedError(\hat{f}_t) = 0.01$
  - $\widehat{w}_t = \frac{1}{2}\ln\left(\frac{1-0.01}{0.01}\right) = \frac{1}{2}\ln(99) = 2.3$
- $WeightedError(\hat{f}_t) = 0.5$
  - $\widehat{w}_t = \frac{1}{2}\ln\left(\frac{1-0.5}{0.5}\right) = \frac{1}{2}\ln(1) = 0$
- $WeightedError(\hat{f}_t) = 0.99$
  - $\widehat{w}_t = \frac{1}{2}\ln\left(\frac{1-0.99}{0.99}\right) = \frac{1}{2}\ln\left(\frac{1}{99}\right) = -2.3$

AdaBoost: Datapoint Weights

- Intuition behind weights
  - α_i should be higher for datapoints that are difficult to classify
- Start by training first model (decision stump) on all data, with $\alpha_i = 1$ for all points
- Then modify your datapoint weights for the next decision stump:
  - $\alpha_{i,t+1} \leftarrow \begin{cases} \alpha_{i,t} e^{-\widehat{w}_t}, & \text{if } \hat{f}_t(x_i) = y_i \\ \alpha_{i,t} e^{\widehat{w}_t}, & \text{if } \hat{f}_t(x_i) \neq y_i \end{cases}$
- Note
  - $e^0 = 1$: if stump t has weight 0 (should not contribute), the datapoint weight will not change
  - $e^{\text{positive value}} > 1$: upweight example
  - $e^{\text{negative value}} < 1$: downweight example
- Question: what if the weight of a tree is negative?

AdaBoost: Datapoint Weights

- Intuition behind weights
  - α_i should be higher for datapoints that are difficult to classify
- Start by training first model (decision stump) on all data, with $\alpha_i = 1$ for all points
- Then modify your datapoint weights for the next decision stump:
  - $\alpha_{i,t+1} \leftarrow \begin{cases} \alpha_{i,t} e^{-\widehat{w}_t}, & \text{if } \hat{f}_t(x_i) = y_i \\ \alpha_{i,t} e^{\widehat{w}_t}, & \text{if } \hat{f}_t(x_i) \neq y_i \end{cases}$
- Note
  - $e^0 = 1$: if stump t has weight 0 (should not contribute), the datapoint weight will not change
  - $e^{\text{positive value}} > 1$: upweight example
  - $e^{\text{negative value}} < 1$: downweight example
- Question: what if the weight of a tree is negative?
- Answer: this still works! A negative weight tree has accuracy worse than random guessing, so we can invert the predictions

One last thing: normalizing weights

- During the AdaBoost algorithm, weights can get very large or very small in magnitude

- To avoid issues with precision, we often normalize the weights:

$$\alpha_{i,t+1} \leftarrow \frac{\alpha_{i,\mathrm{t+1}}}{\sum_{j=1}^{n} \alpha_{j,t+1}}$$

AdaBoost Algorithm

Train

for t in $[1, 2, \ldots, T]$:

- Learn $\hat{f}_t(x)$ based on data weights $\alpha_{i,t}$
- Compute model weight $\widehat{w}_t$
- Compute data weights $\alpha_{i,t+1}$

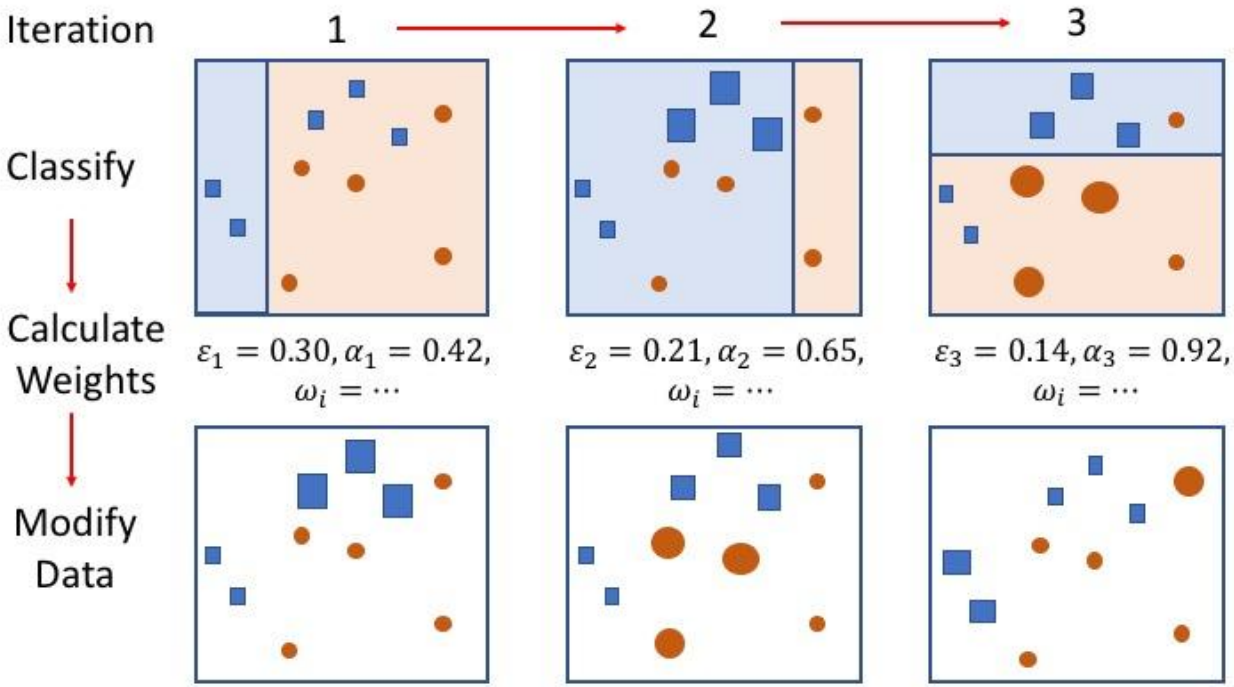
$$\widehat{w}_t = \frac{1}{2}\ln\left(\frac{1 - WeightedError\left(\hat{f}_t\right)}{WeightedError(\hat{f}_t)}\right)$$

$$\alpha_{i,t+1} \leftarrow \begin{cases} \alpha_{i,t}e^{-\widehat{w}_t}, & \text{if } \hat{f}_t(x_i) = y_i \\ \alpha_{i,t}e^{\widehat{w}_t}, & \text{if } \hat{f}_t(x_i) \neq y_i \end{cases}, \alpha_{i,t+1} \leftarrow \frac{\alpha_{i,\mathrm{t}+1}}{\sum_{j=1}^{n}\alpha_{j,t+1}}$$

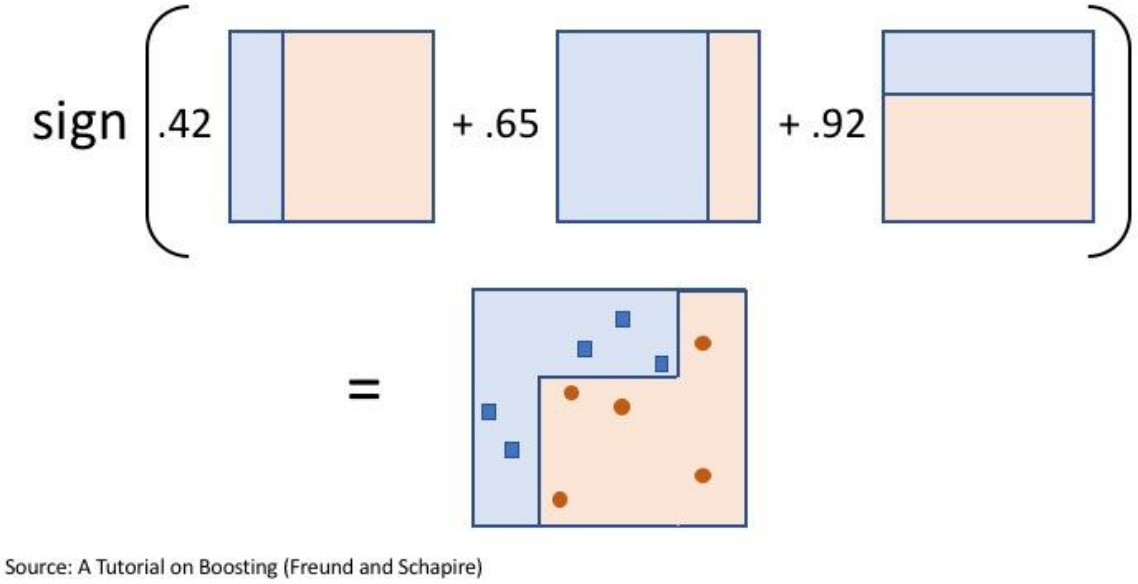
Predict

$$\hat{y} = \hat{F}(x) = sign\left(\sum_{t=1}^{T}\widehat{w}_t\hat{f}_t(x)\right)$$

AdaBoost Algorithm



Source: A Tutorial on Boosting (Freund and Schapire)



Source: A Tutorial on Boosting (Freund and Schapire)

AdaBoost Overfitting

- With enough weak learners that are better than random guessing, AdaBoost can achieve 0 training error
  - As usual, that can mean overfitting

- But in general, AdaBoost is less likely to overfit than many other classification algorithms

- May still want to choose a number of trees that minimizes validation error, rather than adding tons of trees

AdaBoost Pros and Cons

- Pros
  - Powerful: typically performs better than random forest with the same number of trees
  - Resistant to overfitting
  - Simple to implement
- Cons
  - Can be sensitive to outliers
  - Slow to train (have to train each tree in sequence, so adding trees means longer training)
    - Optimized software, like XGBoost from UW, can make training faster

- Other boosting algorithms
  - **Gradient Boosting** uses gradients instead of the AdaBoost weighting system
    - May cover later if there is time
  - Gradient Boosting is often more accurate than AdaBoost, but is more prone to overfitting and has many hyperparameters to tune

One faster way to optimize hyperparameters

- Searching for best parameters can start to take a very long time, especially for models that take a while to train

- Halving Grid Search
  - Start with a small amount of data and evaluate all hyperparameter setting combinations
  - Take a small portion of the best-performing hyperparameter settings (default 1/3 in sklearn)
    - Test this subset of hyperparameter settings on more data
  - Continue until testing on all data

- Not optimal, but often comes close and runs much faster!
- We'll use this in Assignment 2

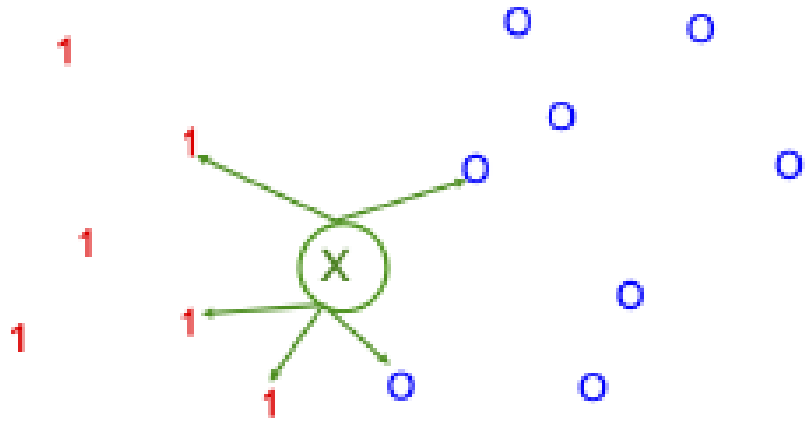
Break

Get some water, stretch your legs, chat, be back in 10

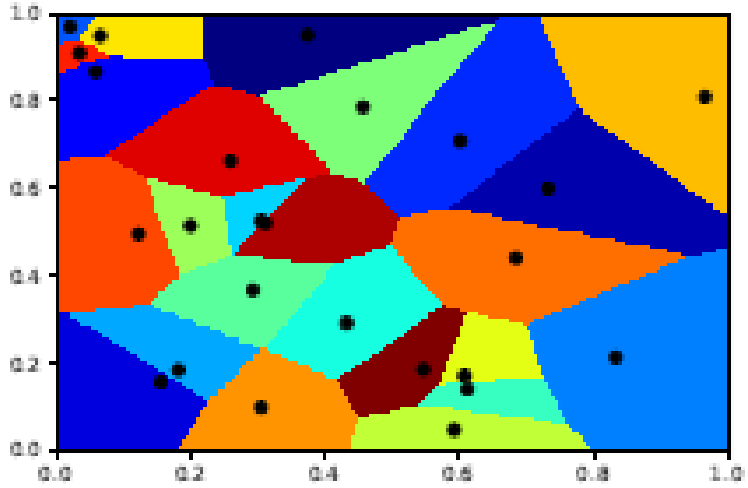
K-Nearest Neighbors

- K-nearest neighbors is one of the simpler models to explain and implement!
- Big idea: if you want to classify a new datapoint, look at **similar** datapoints in your dataset
- Use their labels to label the new datapoint
- No function to learn!

K-Nearest Neighbors



(a)



(b)

Voronoi Tessellation: all points that are "closest" to any training point

- Input
  - x_q: query
  - $x_1, \ldots, x_n$: dataset
- To classify x_q:
  - Find its k nearest neighbors in the training data (smallest distance in feature space)
  - Set $\hat{y}$ to the majority vote of the labels of these nearest neighbors
- **Local Method:** only uses "local" data to make a decision

K-Nearest Neighbors

$\textit{Input:}\ x_q$

$X^{kNN} = [x_1, \ldots, x_k]$

$nn\_dists = [dist(x_1, x_q), dist(x_2, x_q), \ldots, dist(x_k, x_q)]$

$for\ x_i \in [x_{k+1}, \ldots, x_n]$:

$dist = distance(x_q, x_i)$

$if\ dist < \max(nn\_dists)$:

$remove\ largest\ dist\ from\ X^{kNN}\ and\ nn\_dists$

$add\ x_i\ to\ X^{kNN} and\ distance(x_q, x_i)\ to\ nn\_dists$

$\textit{Output:}\ X^{kNN}$

K-Nearest Neighbors

- Yes, it is really that simple
  - But there are many changes and choices you can make!

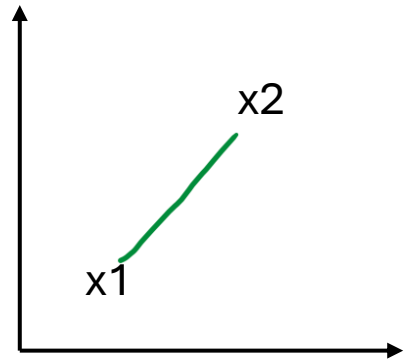
- Complicating design choices
  - Number of nearest neighbors k
  - Distance metric
  - Aggregation method
    - Majority vote, weighted average by distance
    - Average, weighted average for **regression**

(Some) Distance Metrics

- How do we define how “close” two sample vectors are?
  - We need a smaller value when two things are “closer”
  - Larger value when two things are “farther”
- Simplest way: Euclidean Distance

$$distance(x_i, x_q) = \left|\left|x_i - x_q\right|\right|_2$$

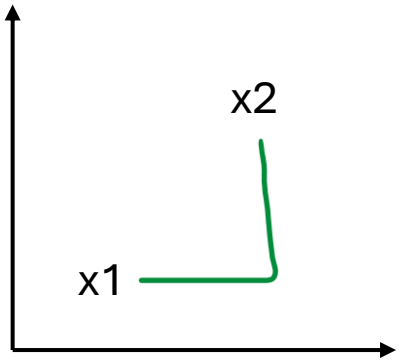
$$= \sqrt{\sum_{j=1}^{D} \left(x_i[j] - x_q[j]\right)^2}$$



- Also: Manhattan Distance

$$distance(x_i, x_q) = \left|\left|x_i - x_q\right|\right|_1$$

$$= \sum_{j=1}^{D} \left|x_i[j] - x_q[j]\right|$$



(Some) Distance Metrics

- If features aren't standardized, some may vary more or be in different units
- We can calculate **weighted distances,** weighting different dimensions differently

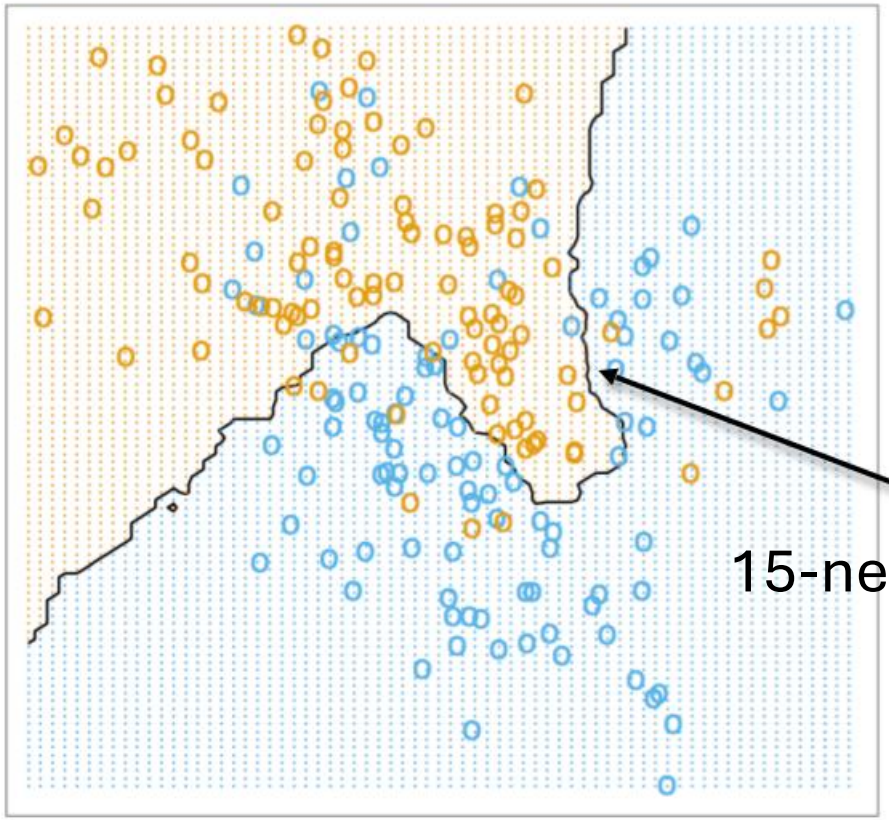
- For feature j:

$$\frac{1}{max_i(x_i[j]) - min_i(x_i[j])} = a_j$$

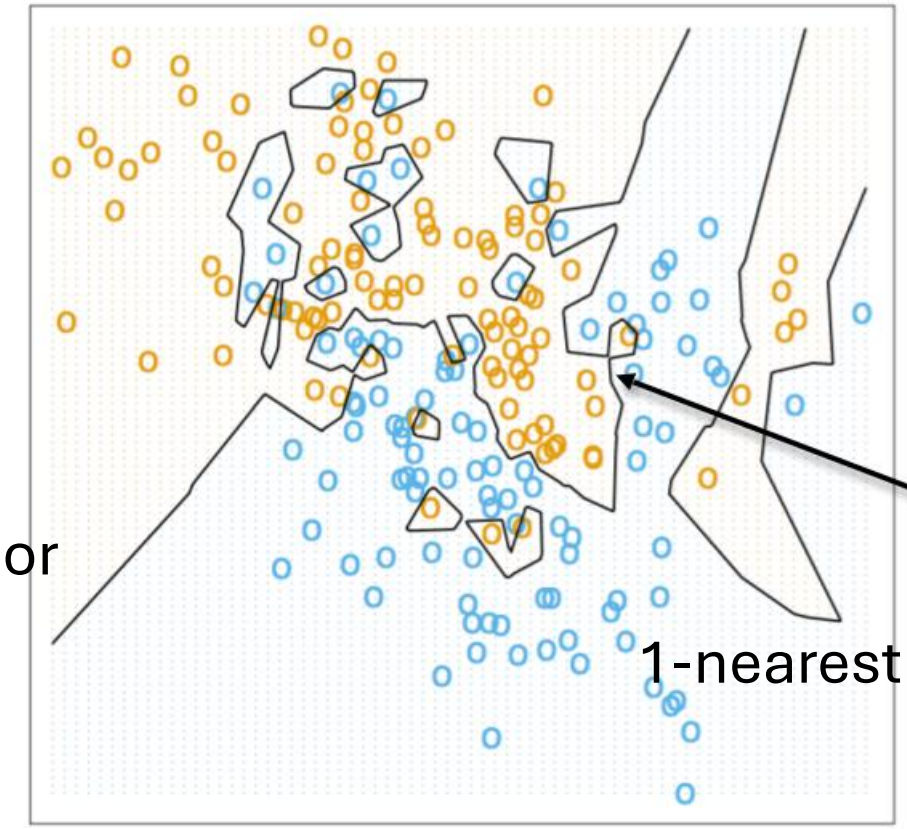
- For Euclidean distance

$$distance(x_i, x_q) = \sqrt{\sum_{j=1}^{D} a_j^2 \left(x_i[j] - x_q[j]\right)^2}$$

Overfitting



15-nearest neighbor



1-nearest neighbor

Figures from Hastie et al

k-NN Pros and Cons

- Pros
  - Easy to implement and explain
  - No training needed (all local)
  - Can classify non-linear data
- Cons
  - Suffers from curse of dimensionality
    - With not enough data/many features, points are very spread out in space
  - Have to keep all data points around to classify new data
  - Function is discontinuous (can make large jumps from small changes in input)
    - Example: large price increase from 1000 to 1001 sq ft house
  - High computation cost during prediction (have to look at all n datapoints and calculate distance)
  - Sensitive to choice of k

Topic shift: nonlinear decision boundaries and kernels

- We've looked at trees and k-NN that can handle non-linear decision boundaries

- Let's go back to one of our previous classification algorithms, SVM, to cover something called **kernels** for non-linear decision boundaries

Nonlinear decision boundaries and kernels

- Original data in 1 dimension: not linearly separable
- Introduce a new feature through a **mapping**: $\phi(x) = \begin{bmatrix} x_1 \\ x_1^2 \end{bmatrix}$
- Now we can find a linear separation in 2 dimensions!

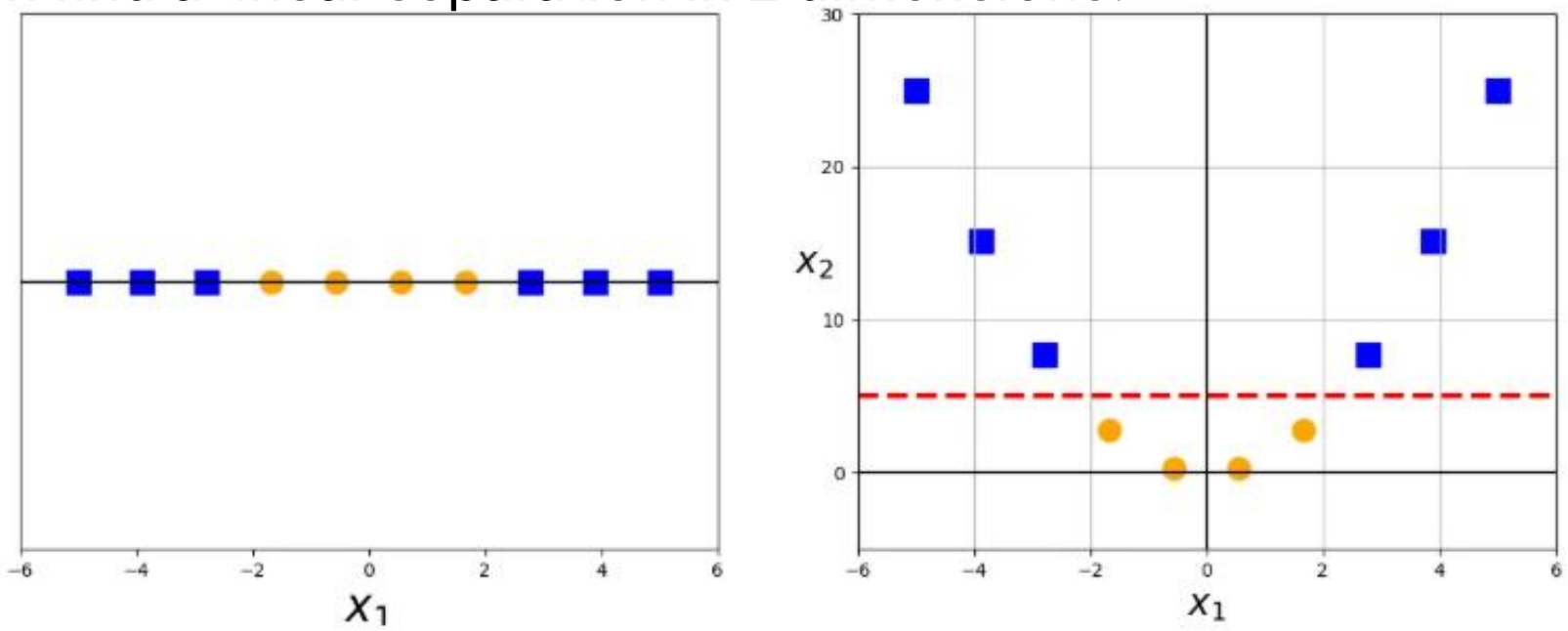


Image from https://medium.com/data-science/the-kernel-trick-c98cdbcaeb3f

Changing the topic: nonlinear data and kernels

- Example in 2d: $\phi(x) = \mathrm{x} \bmod 2$

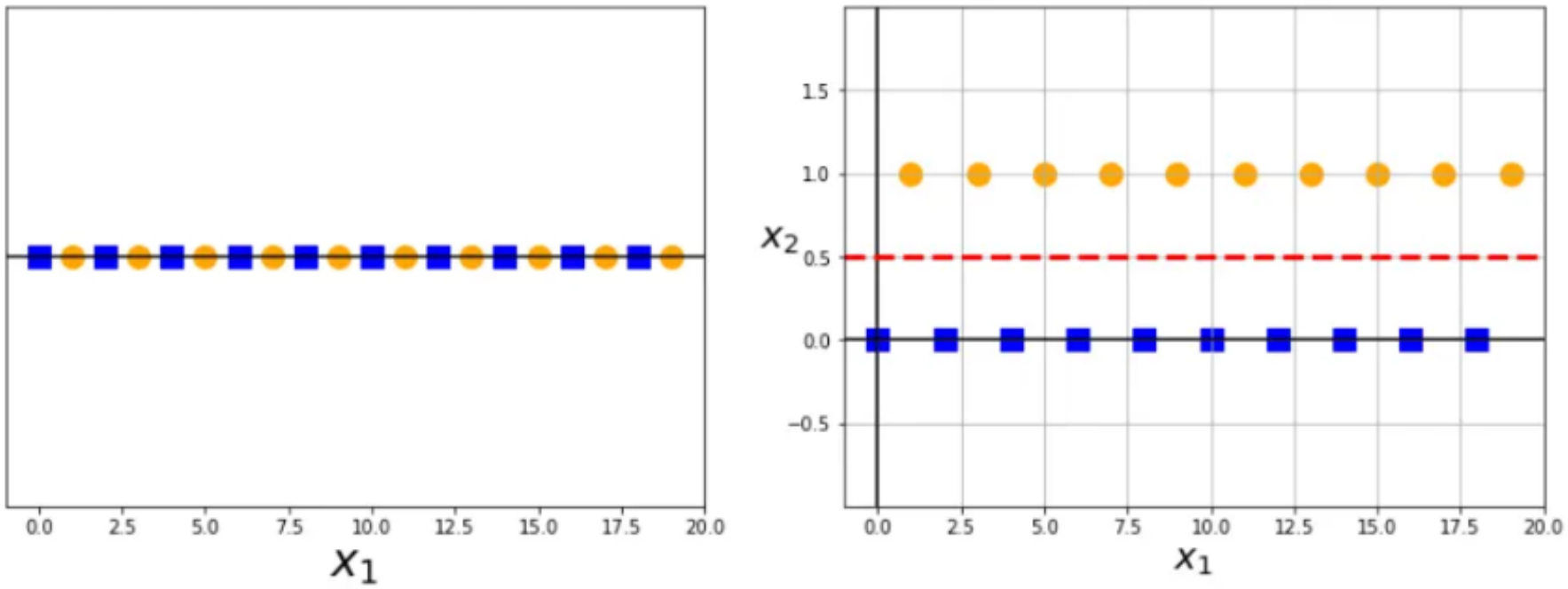


Image from https://medium.com/data-science/the-kernel-trick-c98cdbcaeb3f

Nonlinear data and kernels

- When talking about SVMs, we mentioned that they can also be used for nonlinear data
- This can be done by using the idea that non-linearly separable data can be linearly separable in higher dimensions
- But first, we have to reformulate the SVM solution
- (Note: this involves Calc III; we'll walk through the intuition of why the reformulation works, but the main thing to take away is what a kernel is and why we use them)

Dual formulation for SVM

- We're going to use **Lagrange multipliers** to reformulate our SVM
- Basic idea behind Lagrange multipliers:
  - You can turn a constrained optimization problem into an unconstrained optimization problem
  - And remember, our SVM solution is a constrained optimization problem!
  - $\min_{w,\mathrm{b}} \frac{1}{2}\|w\|_2^2$ subject to constraints: $y_i(\boldsymbol{w}^T x_i + \mathrm{b}) \geq 1$
    - Assuming the two classes are -1 and 1
- Lagrange multipliers: when your optimizing function is minimized with constraints, the gradient of the optimizing function is **parallel** to the gradient of the constraint function

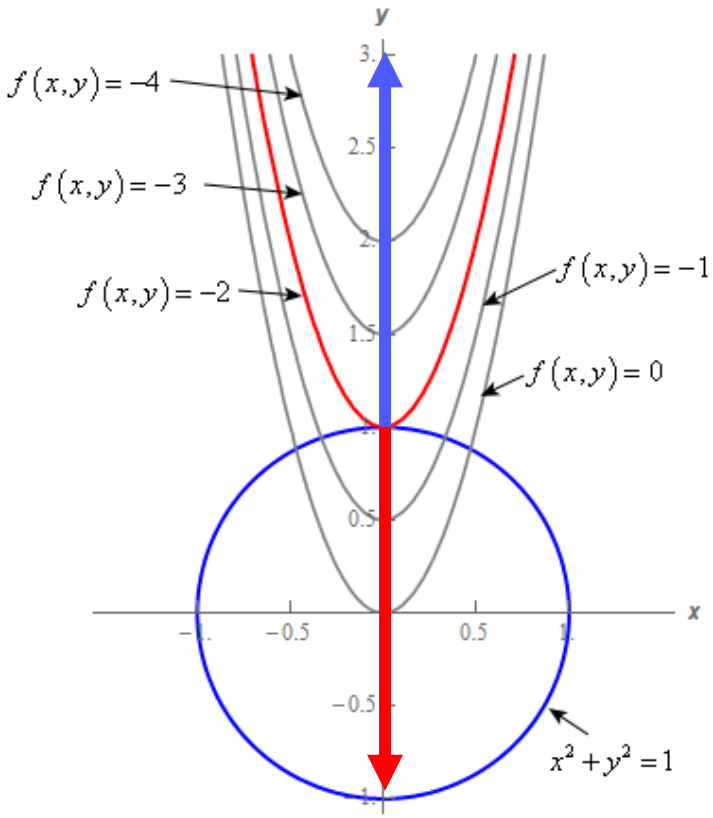


Image from https://tutorial.math.lamar.edu/classes/calciii/lagrangemultipliers.aspx

Dual formulation for SVM

- If your optimizing function is $f(x)$, and your constraint function is $g(x), = 0$ the fact that their gradients are parallel means that

$$\nabla f = \alpha \nabla g$$

- We call α here a **Lagrange multiplier**

- To find the x that minimizes $f(x)$, we can find

$$min_x(max_\alpha(f(x) - \alpha g(x))$$

- Intuition behind why we can use this new function
  - $max_\alpha(f(x) - \alpha g(x) = \text{f(x) when } g(x), = 0$
  - $max_\alpha(f(x) - \alpha g(x) = \infty \text{ when } g(x), \neq 0$
  - So this just lets us find the minimum of f(x) subject to our original constraint

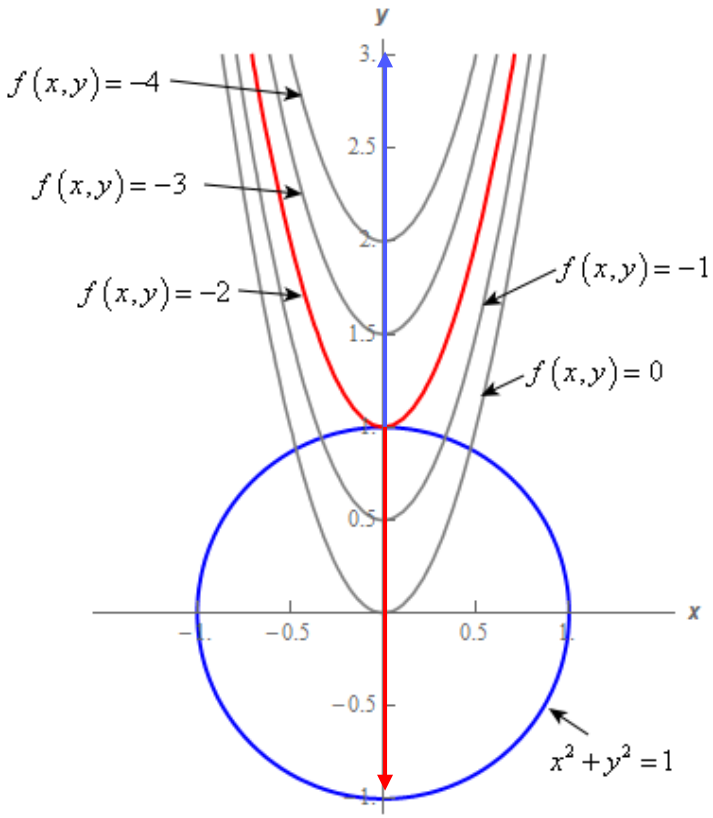


Image from https://tutorial.math.lamar.edu/classes/calciii/lagrangemultipliers.aspx

Dual formulation for SVM

- Translating this into the SVM:
  - $\min_{w,\mathrm{b}} \frac{1}{2}\|w\|_2^2$ subject to constraints: $y_i(\boldsymbol{w}^T x_i + \mathrm{b}) \geq 1$
  - Goes to $\mathrm{L}(\mathrm{w}, \mathrm{b}, \alpha) = min_{w,\mathrm{b}} max_{\alpha} \frac{1}{2}\|w\|_2^2 - \sum_{i=1}^{n} \alpha_i[(\boldsymbol{w} \cdot \boldsymbol{x_i} + \mathrm{b})y_i - 1]$
    - $\boldsymbol{w} \cdot \boldsymbol{x_i}$: dot product, equivalent here to $\boldsymbol{w}^T x_i$
  - We can reformulate as $max_{\alpha} min_{w,\mathrm{b}} \frac{1}{2}\|w\|_2^2 - \sum_{i=1}^{n} \alpha_i[(\boldsymbol{w} \cdot \boldsymbol{x_i} + \mathrm{b})y_i - 1]$
    - Flip min and max
- To obtain the minimum with respect to w and b, we must find where the partial derivative with respect to w and b are 0
  - $\frac{\partial \mathrm{L}}{\partial \mathrm{w}} = w - \sum_{i=1}^{n} \alpha_i y_i x_i = 0$
  - $w = \sum_{i=1}^{n} \alpha_i y_i x_i$
  - $\frac{\partial \mathrm{L}}{\partial \mathrm{b}} = -\sum_{i=1}^{n} \alpha_i y_i = 0$
  - $\sum_{i=1}^{n} \alpha_i y_i = 0$

Dual formulation for SVM

- We'll skip ahead a few steps for time, but now that we've rewritten w and b, we can find the following formulation of our original problem:

$$max_{\alpha} \frac{1}{2} \sum_{i=1}^{n} \sum_{\mathrm{j}=1}^{n} \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^{n} \alpha_{\mathrm{i}}$$

- Once you solve this **dual formulation**, the decision for a new input x is given by the sign of:

$$\sum_{i=1}^{n} \alpha_{\mathrm{i}} \, y_i (x_i \cdot x) + b$$

- b can be found by using any support vector x_i, which satisfies

$$y_i (w^T x_i - b) = 1$$

- If we're working with high-dimensional data to try to transform into a linearly-divisible space, we can use a map $\phi(x)$ instead of x in these equations
- But computing and storing high-dimensional data and then computing dot products is computationally expensive!

Kernel Trick

- Ok, so why did we rewrite this problem in the first place?
- We're going to use something called the **kernel trick**
- $K(x, x')$: $\mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ is a **kernel** for a map $\phi(x)$ if $K(x, x') = \phi(x) \cdot \phi(x')$
- If we can represent models as functions of dot products of feature maps, and we can find a kernel for that feature map such that $K(x, x') = \phi(x)^T \phi(x')$, we can avoid explicitly storing and computing high-dimensional maps!

Kernel Trick

- Now we can use kernels in our SVM for nonlinear data!
- Instead of

$$max_{\alpha} \frac{1}{2} \sum_{i=1}^{n} \sum_{\mathrm{j}=1}^{n} \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^{n} \alpha_{\mathrm{i}}$$

- We'll use

$$max_{\alpha} \frac{1}{2} \sum_{i=1}^{n} \sum_{\mathrm{j}=1}^{n} \alpha_i \alpha_j y_i y_j \mathrm{K}(x_i, x_j) - \sum_{i=1}^{n} \alpha_{\mathrm{i}}$$

- Where $\mathrm{K}(x_i, x_j)$ computes the dot product of a mapping for x
- And the decision for a new input x can be found by the sign of

$$\sum_{i=1}^{n} \alpha_{\mathrm{i}} \, y_i \mathrm{K}(x_i, x) + b$$

Dual formulation for SVM

- Example:

$$\phi(x) = \begin{bmatrix} x[1]^2 \\ \sqrt{2}\mathrm{x}[1]\mathrm{x}[2] \\ x[2]^2 \end{bmatrix}, K(x, x') = (x^T x')^2$$

- Consider $x = [1,2], x' = [3,4]$
- Show that the output of the mapping and the kernel are the same
- Reminder of $x^T x' = x[1]x'[1] + x[2]x'[2]$

Dual formulation for SVM

- Example:

$$\phi(x) = \begin{bmatrix} x[1]^2 \\ \sqrt{2}\mathrm{x}[1]\mathrm{x}[2] \\ x[2]^2 \end{bmatrix}, K(x, x') = (x^T x')^2$$

- Consider $x = [1,2], x' = [3,4]$
- $\phi(x)^T \phi(x) = [1, \sqrt{2} * 2, 4] \begin{bmatrix} 9 \\ \sqrt{2} * 12 \\ 16 \end{bmatrix} = 9 + 2 * 24 + 64 = 121$
- $K(x, x') = [1,2] \begin{bmatrix} 3 \\ 4 \end{bmatrix}^2 = (3 + 8)^2 = 121$

- Kernels are *much* faster than explicitly computing maps!

Popular Kernels

- Polynomials of degree k
  - $K(x, x') = (x^T x')^k$
- Gaussian kernel (RBF kernel, Radial Basis Function)
  - $K(x, x') = exp\left(-\frac{\|x\text{-}x'\|_2^2}{2\sigma^2}\right)$
- Sigmoid kernel
  - $K(x, x') = \tanh(\gamma x^T x' + r)$

Demo

Bias and Fairness Discussion

Discuss some of the potential biases and fairness concerns that might be present in our dataset about churn